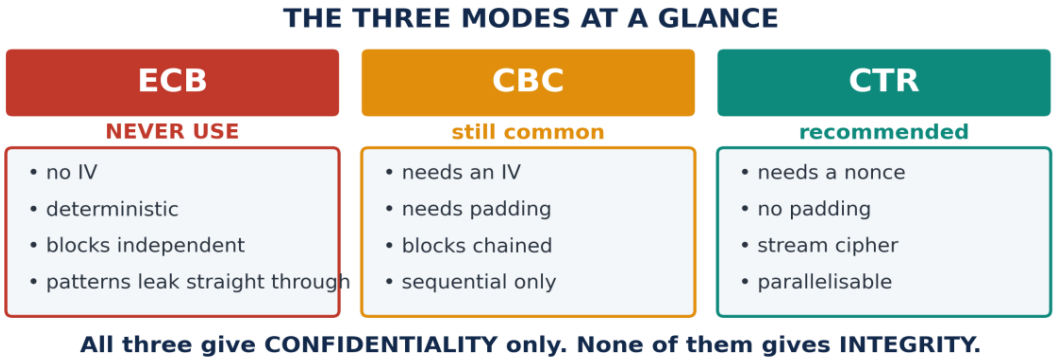
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY)

Institute of Technical Education and Research (ITER) • Bhubaneswar, Odisha

CHAPTER 3

Symmetric Encryption: Two Sides, One Key

A Complete, Illustrated, Line-by-Line Tutorial for B.Tech Students



Prescribed textbook

Seth James Nielson • Christopher K. Monson

Practical Cryptography in Python: Learning Correct Cryptography by Example

Apress, 2019 • Chapter 3, pages 53–110 • https://doi.org/10.1007/978-1-4842-4900-0_3

How to Use This Tutorial

As in the Chapter 1 and Chapter 2 companions, dark blue chips such as tell you which printed page of Chapter 3 the material that follows corresponds to. The chips run in order from page 53 to page 110.

Everything in this document was executed

Cryptography tutorials that print un-run code are worse than useless. Every listing in this document was typed into a Python 3 interpreter with the cryptography module installed and actually run; the outputs shown are what came back. The NIST known-answer tests pass 4/4. The encrypted images in Section 13 are not illustrations they are real AES output, produced by the very program the chapter asks you to write.

What you need before you start

- The virtual environment from Chapter 1, with the cryptography module installed. Nothing new is required.
- Chapters 1 and 2 completed. Chapter 3 assumes you know what a key is, what YANAC means, and why os.urandom matters.
- An image editor (or a few lines of Python) to make a bitmap for the ECB experiment.

Course learning outcomes for this chapter

1. Explain what “symmetric” means and how a symmetric cipher differs from the substitution cipher of Chapter 1.
2. Use the cryptography module to build AES encryptor and decryptor objects, and explain the update()/finalize() contract.
3. State the three security properties confidentiality, integrity, authentication and identify which one encryption alone provides.
4. Describe ECB, CBC and CTR modes, and demonstrate visually why ECB must never be used.
5. Apply XOR fluently, including its inversion property, and derive the CBC decryption formula.
6. Use initialisation vectors and padding correctly, and explain what goes wrong when a key/IV pair is reused.
7. Mount practical attacks: ciphertext splicing, keystream theft, message forgery and the SSLv3 padding oracle.
8. Generate keys from a cryptographically secure source and explain why random.seed() is a catastrophe.

A map of the chapter

Book pages	What happens	Listings
53–57	Alice and Bob meet AES and stumble into ECB mode	3-1
57–60	What encryption really means; AES as a block cipher; the three modes	
60–63	ECB explained and validated against the NIST known-answer tests	3-2

Book pages	What happens	Listings
64–70	ECB broken: repeated blocks, splicing, and the TOP SECRET image	3-3, 3-4
70–74	IVs, XOR, and the CBC chain	
75–85	CBC in code, PKCS7 padding, the broken manager and the fixed one	3-5 to 3-8
86–91	Counter mode turning a block cipher into a stream cipher	3-9
91–96	Key and IV management; what reuse costs you	3-10
96–99	Malleability: changing a message you cannot read	
99–106	The SSLv3 padding oracle attack, built from scratch	3-11 to 3-13
107–110	Weak keys, other ciphers, and finalize()	

BOOK PAGE 53

1. Let's Scramble!

Symmetric encryption is at the foundation of all modern secure communications. It is what we use to scramble messages so that people can only decrypt them if they have access to the same key used to encrypt them. That is what "symmetric" means in this case: one key is used on both ends of the communication channel, to both encrypt and decrypt messages.

1.1 The story so far

Unsurprisingly, the villains of East Antarctica are at it again, causing all kinds of trouble for their neighbours. This time, Alice and Bob are spying on the enemy troops to the west, doing reconnaissance on the size of their snowballs and the accuracy of their throws.

In earlier missions, Alice and Bob used the Caesar cipher from Chapter 1 to protect their messages. As you discovered, that cipher is easy to crack. As a result, the East Antarctica Truth-Spying Agency (EATSA) has equipped them with modern cryptography that uses a key to encode and decode secret messages. This new technology belongs to a class of encryption algorithms called symmetric ciphers, because both the encryption and decryption processes use the same shared key. The specific algorithm they are using in this post-et-tu-Brute world is the Advanced Encryption Standard (AES).

FOOTNOTES 1 AND 2

"Villains" or heroes, depending on your point of view, Padawan.

The name "Advanced Encryption Standard" is actually more of a title. Many algorithms competed to become the Advanced Encryption Standard, including many that are still available today. The original name of the winning algorithm was Rijndael, a composite of the last names of its two inventors.

	Chapter 1 shift cipher	Chapter 3 AES
Key	a number from 1 to 25	128, 192 or 256 random bits
Key space	25	2^{128} or more
Unit of operation	one character	one 16-byte block
Same key both ways?	yes also symmetric	yes that is the definition
Broken by	a seven-line loop	nothing practical, if used correctly

BOOK PAGE 54

2. Making a Key, and a Bad Choice of Mode

Alice and Bob do not have a lot of information about the proper care and handling of AES. They have just enough documentation to get encryption and decryption working.

“The docs say we have to create AES keys,” Alice says, holding one of the manuals. “Apparently, it’s fairly easy. We have sample code here.”

```
1 import os
2 key = os.urandom(16)
```

“Wait… really?” Bob asks. “That’s it?”

Alice is right: that is all it takes. An AES key is just random bits 128 of them (16 bytes’ worth) in this example. This will allow us to use AES-128.

> **WHY os.urandom AND NOT random.random**
>
> os.urandom asks the operating system for bytes from its cryptographically secure random source. It is the same function you used for salts in Chapter 2, and for exactly the same reason.
>
> A key is only as strong as the randomness behind it. Section 32 of this tutorial returns to this point and shows what happens when a key is generated from a predictable seed the entire cipher collapses no matter how strong AES is.

2.1 Choosing a mode badly

With the random key created, how do we then encrypt and decrypt messages? Earlier, we used the Python cryptography module to create hashes. It does many other things as well. Let’s see how Bob encouraged by the ease of creating keys uses it now to encrypt messages with AES.

Bob takes the documentation from Alice and looks at the next section, noting that there are many different modes of AES computation. Having to choose between them sounds a bit overwhelming, so Bob picks the one that looks easiest to use.

“Let’s use ECB mode, Alice,” he says, looking up from the docs.

“ECB mode? What is that?”

“I don’t really know, but this is the Advanced Encryption Standard. It should all work fine, right?”

> **WARNING ECB: NOT FOR YOU**
>
> We’re going to find out later that ECB mode is terrible and should never be used. But we’ll just follow along for now.
>
> Bob’s reasoning “it’s the standard, so it must be fine” is one of the most common and most dangerous mistakes in applied cryptography. The algorithm being strong tells you nothing about whether the mode you wrapped around it is safe.

2.2 Listing 3-1 AES ECB Code

Here is the code they used to create an “encryptor” and a “decryptor”.

Listing 3-1. AES ECB Code

```
1  # NEVER USE: ECB is not secure!
2  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
3  from cryptography.hazmat.backends import default_backend
4  import os
5
6  key = os.urandom(16)
7  aesCipher = Cipher(algorithms.AES(key),
8            modes.ECB(),
9            backend=default_backend())
10 aesEncryptor = aesCipher.encryptor()
11 aesDecryptor = aesCipher.decryptor()
```

Line by line

Line(s)	What that line actually does
1	A comment the authors put at the top of every ECB listing in the chapter. Take it literally.
2	Imports three things from the module’s cipher package: `Cipher` (the container), `algorithms` (which supplies AES) and `modes` (which supplies ECB, CBC, CTR and the rest). Note the word **`hazmat`** in the path “hazardous materials”, the library authors warning you that these are sharp low-level tools.
3	The backend is the engine that does the real work, normally OpenSSL. The computation is not performed in Python, which makes it both fast and well tested.
4	Needed for `os.urandom`.
6	A fresh 128-bit key. 16 random bytes and nothing more.
7 – 9	Builds the cipher context. Read the three arguments carefully: the **algorithm** takes the key, the **mode** takes its own parameters (ECB needs none), and the backend does the work. This separation is the whole design of the API.
10 – 11	Produces the two working objects. Note they come from the *same* context, so they share the same key which is exactly what “symmetric” means.

BOOK PAGE 55

“That’s not so bad,” Alice says. “What happens now?”

“Apparently, both the encryptor and decryptor have an update method. That’s pretty much it. The encryptor’s update returns the ciphertext.”

EXERCISE 3.1 A SECRET MESSAGE

Without looking at additional documentation, try to figure out how the aesEncryptor.update() and aesDecryptor.update() methods work. Hint: you are going to get some unexpected behaviour, so try lots of inputs. Consider starting with b"a secret message" and then decrypting the result.

3.1 The confusing interactive session

Alice and Bob start trying to figure out the update method. Perhaps inspired by the previous chapter on hashing, where they hashed their names, they try encrypting their names in an interactive Python shell. Alice goes first.

REPRODUCE THIS EXACTLY

The AES example code here uses the key b"\x81\xff9\xa4\x1b\xbc\xe4\x84\xec9\x0b\x9a\xdbu\xc1\x83" in case you want to get identical results. Every byte of output below was reproduced with that key before being printed here.

the session, exactly as it runs

```
>>> aesEncryptor.update(b'alice')
b''
>>> aesEncryptor.update(b'bob')
b''
>>> aesEncryptor.update(b'bob')
b''
>>> aesEncryptor.update(b'bob')
b''
>>> aesEncryptor.update(b'bob')
b'\xe7\xf9\x19\xe3!\x1d\x17\x9f\x80\x9d\xf5\xa2\xbaTi\xb2'
>>> aesDecryptor.update(_)
b'alicebobbobbobbo'
```

“I didn’t get any ciphertext,” Alice grumbles. “What did I do wrong?” “I don’t know. Let me try,” Bob responds. Out of frustration he tries it several more times, and suddenly something comes out. “Wait!” Alice stops him. “You got something!” “Weird! I didn’t do anything different. What happened?” “Now try decrypting it,” Alice suggests.

3.2 What actually happened

Playing around a bit more, and re-reading the docs, Alice and Bob learn what you already discovered from the exercise: the update functions for both encryption and decryption always work on 16 bytes at a time. Calling update with fewer than 16 bytes produces no immediate result. Instead, it accumulates data until it

has at least 16 bytes to work with. Once 16 or more bytes are available, as many 16-byte blocks of ciphertext as possible are produced.

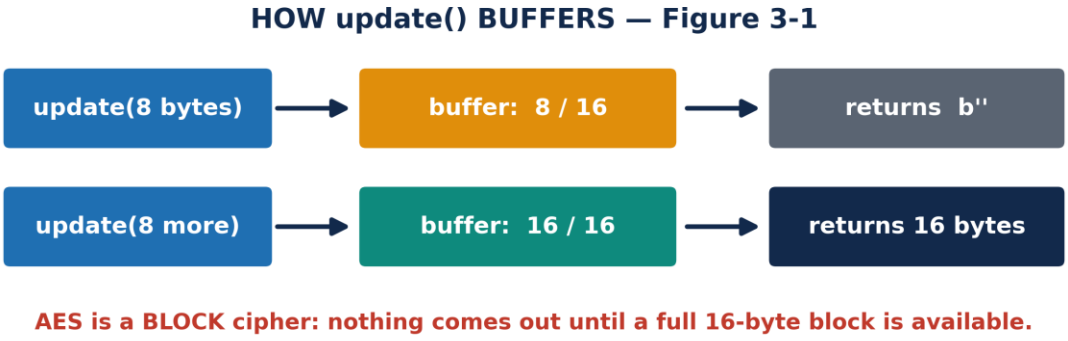


Figure 3.1 Two calls to the update method. The first 8 bytes return nothing because there is not a full block of data to encrypt yet. (Book Figure 3-1.)

Count the bytes and the mystery evaporates

Call	Bytes in	Buffer after	Returned
update(b'alice')	5	5	b''
update(b'bob')	3	8	b''
update(b'bob')	3	11	b''
update(b'bob')	3	14	b''
update(b'bob')	3	17 → emit 16, keep 1	one 16-byte block

The decrypted block is `b'alicebobbobbobbo'` 5 + 3 + 3 + 3 + 3 = 17 bytes fed in, the first 16 of which are “alicebobbobbobbo”. The seventeenth byte, the final “b”, is still sitting in the buffer.

> **THE RULE THAT GOVERNS THIS ENTIRE CHAPTER**
>
> The number of calls to update() is irrelevant. What is encrypted is not the input to any one call, but the concatenation of every input across all calls, plus whatever finalize() produces at the end.
>
> Hold on to this sentence. It explains the surprising output of Listing 3-6, the bug in Listing 3-7, and the design of Listing 3-8.

BOOK PAGE 57

EXERCISE 3.2 UPDATED TECHNOLOGY

Upgrade the Caesar cipher application from Chapter 1 to use AES. Instead of specifying a shift value, figure out how to get keys in and out of the program. You will also have to deal with the 16-byte message size issue. Good luck!

4. What Is Encryption, Really?

For those who have heard of cryptography, encryption is probably what they have heard about most. Web sites and online services will often mention encryption to reassure you that your information is secure. They will typically include phrases like “All data transmitted over the Internet is protected by 128-bit encryption, preventing theft.”

Don’t you feel better already?

Statements like that are really just marketing. They sound nice, but do not usually mean much. That is because “encryption” includes easy-to-break things like Caesar ciphers, and it also is not enough by itself to make communications secure. In cryptography there are several properties that contribute to different aspects of security, and they need to work together. These properties are commonly viewed as the most critical:

1. Confidentiality
2. Integrity
3. Authentication

4.1 Confidentiality

The encryption we explore in this chapter is all about confidentiality. Confidentiality means that only folks with the right key are able to read the data. We use encryption to protect messages so that outsiders cannot read them.

4.2 Integrity

Equally important is integrity. Integrity means that the data cannot be changed without you noticing. It is critical to understand that just because something cannot be read does not mean it cannot be usefully altered. To drive that point home, we are going to do exactly that sort of mischief in this chapter.

4.3 Authentication

Finally, authentication relates to knowing the identity of the party with whom you are communicating. Authentication typically includes some mechanism to establish identity and presence, as well as the ability to tie communication to the established identity.

FOOTNOTE 3 identity and presence

Identity and presence mean loosely "I know who this is, and I know that they consent to my knowing that right now." If you have ever had to dig out your credit card to provide the CVV code to a web site that already has your card on file, you have run into the concept of presence: the CVV code is meant to be an indication that your card is there with you, and therefore that you are around to consent to its use.

4.4 Why all three are needed

Hopefully it is obvious that all three of these properties are essential in many forms of communication. Confidentiality will do Alice and Bob little good if Eve can change what the messages actually say without them knowing: Eve does not need to read the messages to cause real problems. Likewise, Alice and Bob will have little success in their covert communications if they are not sure they have the right person on the other end of the channel.

Keep these ideas in mind as you go through this chapter. Our focus on confidentiality is useful for presentation and it is indeed a critical component of security, but it is not enough. Spending some time with confidentiality by itself will help us to demonstrate how inadequate it is without its friends.

Property	Question it answers	Does AES alone give it?
Confidentiality	Can anyone else READ this?	Yes that is what this chapter builds
Integrity	Has anyone CHANGED this?	NO Sections 27 to 31 exploit exactly this gap
Authentication	Am I talking to the right party?	NO Chapters 5 and 6 address it

BOOK PAGE 58

5. AES: A Symmetric Block Cipher

As mentioned before, the idea behind symmetric encryption is that the same key is used for both encryption and decryption. In the real world, almost all keys to physical locks can be thought of as symmetric: the same key that locks your door also unlocks it. There are other extremely important approaches to encryption that use distinct keys for each operation, but we will get to those in later chapters.

5.1 Block ciphers and stream ciphers

Symmetric key encryption algorithms are often divided into two subtypes: block ciphers and stream ciphers. A block cipher gets its name from the fact that it works on blocks of data: you have to give it a certain amount of data before it can do anything, and larger data must be broken down into block-sized chunks (also, every block must be full). Stream ciphers, on the other hand, can encrypt data one byte at a time.

	Block cipher	Stream cipher
Unit of work	a fixed-size block 16 bytes for AES	one byte, or even one bit
Partial data	must be padded to a full block	no padding needed at all
Example	AES in ECB or CBC mode	AES in CTR mode; historically RC4
Where you met it	the b'' mystery of Section 3	Section 22 of this tutorial

BOOK PAGE 59

5.2 Why AES and only AES

AES is fundamentally a symmetric key, block cipher algorithm. It is not the only one by any stretch, but it is the only one that we will pay any attention to here. It is used in many common Internet protocols and operating system services, including TLS (used by HTTPS), IPSec, and file-level or full-disk encryption. Given its ubiquity, it is arguably the most important cipher to know how to use properly. More importantly, the principles of correct use of AES transfer easily to correct use of other ciphers.

Finally, even though AES is essentially a block cipher, it (like many other block ciphers) can be used in a way that makes it behave like a stream cipher, so we do not lose any teaching opportunities by excluding native stream ciphers from the discussion. In the past, RC4 was a commonly used stream cipher, but it has been found vulnerable to various attacks and is being replaced by stream modes of AES.

Also, as Bob says, “It’s advanced!” That ought to be enough for anyone, right?

EXERCISE 3.3 HISTORY LESSON

Do some research online about DES and 3DES. What is the block size for DES? What is its key size? How does 3DES strengthen DES?

EXERCISE 3.4 OTHER CIPHERS

Do a little research about RC4 and Twofish. Where are they used? What kinds of problems does RC4 have? What are some of Twofish’s advantages over AES?

5.3 The block size, deduced

Since AES is a pretty good place to start, let’s dig in with a little bit of background. We know it is a symmetric key block cipher. Given what we saw of Alice’s and Bob’s attempts to use it, can you guess the block size?

If you were thinking “16 bytes!” (128 bits), you get a gold star. Tell all your friends over an encrypted channel.

A DISTINCTION STUDENTS CONSTANTLY BLUR

The AES BLOCK size is always 128 bits, for every variant of AES. The AES KEY size may be 128, 192 or 256 bits. These are two different numbers and changing the key size does not change the block size.

This is why an AES-CBC initialisation vector is always 16 bytes even when the key is 32 bytes the IV is XORed with a block, not with a key.

BOOK PAGE 60

6. The Three Modes of Operation

AES has several modes of operation that allow us to achieve different cryptographic properties:

1. Electronic code book (ECB) WARNING! DANGEROUS!
2. Cipher block chaining (CBC)
3. Counter mode (CTR)

These are not the only modes of operation for AES. In fact, while CBC and CTR are still used, a newer mode called GCM is now recommended to replace them in many circumstances, and we will examine GCM in detail later in this book. These three modes are, however, very instructive, and together they cover the most important concepts. They will provide a solid foundation on which to build greater understanding of block ciphers in general and AES in particular.

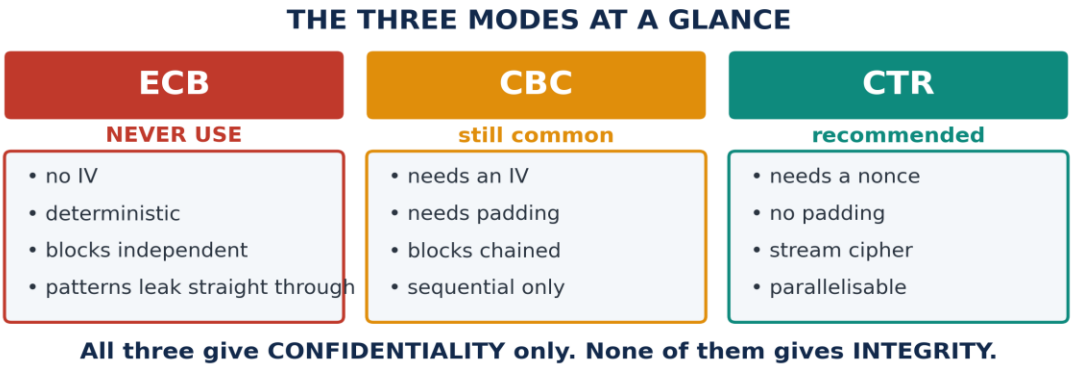


Figure 6.1 The three modes covered in this chapter. Keep this page bookmarked; the rest of the chapter fills in the detail.

7. ECB Is Not for Me

Be warned: relying on ECB mode for security is irresponsibly dangerous and it should never be used. Think of it as being good for testing and educational purposes only. Please, do not ever use it in your applications or projects. Seriously. You have been warned. Don't make us come over there.

> **THE PATTERN YOU SHOULD BE NOTICING BY NOW**
>
> Sometimes the best approaches for explaining a thing are not at all suitable for using it in practice. This seems to apply particularly well to cryptography, which is one reason that the authors urge people to always use a well-established library instead of building their own.
>
> The basic principles are simple, but without all of the complex trappings that come with mature libraries and a solid understanding of how to use them, those principles alone will give you very poor security, not just "slightly imperfect" security. There is rarely much in the way of middle ground; once the safe is cracked, it doesn't matter how thick its walls are. Cryptographic concepts are often simple, but safe and correct implementation is usually complex.

BOOK PAGE 61

7.1 What ECB actually is

With all of those warnings out of the way (not really, there will be more), what is ECB? In a way, ECB is “raw” AES: it treats every 16-byte block of data independently, encrypting each one in exactly the same way using the provided key. As we will see with counter mode and cipher block chaining mode, there are a lot of interesting ways to use that approach as a building block for a more advanced and secure cipher, but it is really not a good way to go about encryption by itself.

The name “electronic code book” hearkens back to earlier days of cryptographic code books, where you would take your (small) key, go to the right page in the book, and use the table on that page to look up the output (ciphertext) that corresponded to each part of your input (plaintext). AES ECB mode can be thought of in that way, but with a mind-bogglingly huge book. The key similarity is that once you have the key, every possible block’s encrypted value is known, and the same is true for decryption; it is like we are looking them up.

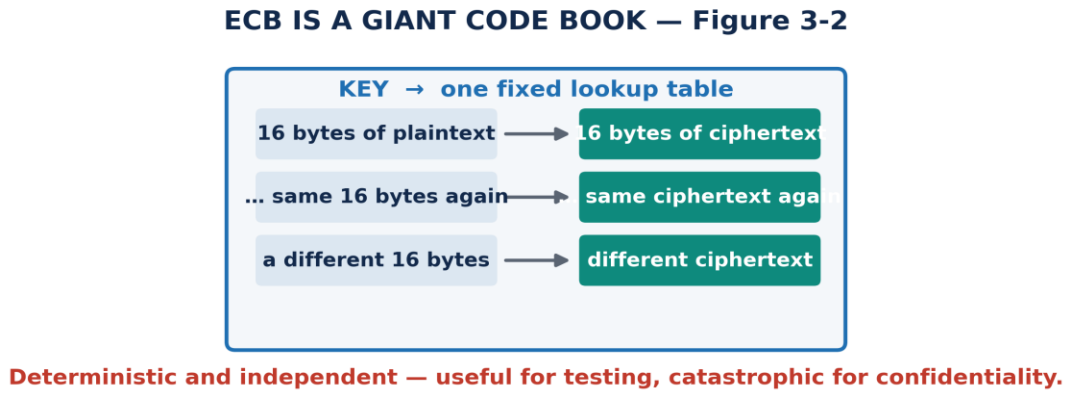


Figure 7.1 ECB mode is analogous to having a big dictionary of plaintext to ciphertext. Every 16 bytes of plaintext has a corresponding 16-byte output. (Book Figure 3-2.)

7.2 The one legitimate use: known-answer tests

As we will see, the properties of determinism and independence are useful but not sufficient properties for message security. ECB mode is useful because it can be used for testing for example, to make sure that the AES algorithm is behaving as expected. Some systems will pick a special key, say all zeros, as a test key. As part of a self-test, the system will run AES in ECB mode with the test key to see if it encrypts as expected. You will sometimes see tests of this kind called KATs (known answer tests).

The National Institute of Standards and Technology (NIST) in the United States publishes a list of KATs that are used for implementation validation. You can download a zip file with these KATs from **https://csrc.nist.gov/CSRC/media/Projects/Cryptographic-Algorithm-Validation-Program/documents/aes/KAT_AES.zip**. That archive contains response (.rsp) files that identify expected outputs for given inputs.

BOOK PAGE 62

7.3 The NIST test vectors

For example, in the ECBGFSbox128.rsp file, the first four ENCRYPT entries are:

ECBGFSbox128.rsp (first four ENCRYPT entries)

```
COUNT = 0
KEY = 00000000000000000000000000000000
PLAINTEXT = f34481ec3cc627bacd5dc3fb08f273e6
CIPHERTEXT = 0336763e966d92595a567cc9ce537f5e

COUNT = 1
KEY = 00000000000000000000000000000000
PLAINTEXT = 9798c4640bad75c7c3227db910174e72
CIPHERTEXT = a9a1631bf4996954ebc093957b234589

COUNT = 2
KEY = 00000000000000000000000000000000
PLAINTEXT = 96ab5c2ff612d9dfaae8c31f30c42168
CIPHERTEXT = ff4f8391a6a40ca5b25d23bedd44a597

COUNT = 3
KEY = 00000000000000000000000000000000
PLAINTEXT = 6a118a874519e64e9963798a503f1d35
CIPHERTEXT = dc43be40be0e53712f7e2bf5ca707209
```

That seems useful. Let's test that theory using Listing 3-2.

7.4 Listing 3-2 AES ECB KATs

Listing 3-2. AES ECB KATs

```
# NEVER USE: ECB is not secure!
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

# NIST AES ECBGFSbox128.rsp ENCRYPT Kats
# First value of each pair is plaintext
# Second value of each pair is ciphertext
nist_kats = [
  ('f34481ec3cc627bacd5dc3fb08f273e6',
   '0336763e966d92595a567cc9ce537f5e'),
  ('9798c4640bad75c7c3227db910174e72',
   'a9a1631bf4996954ebc093957b234589'),
  ('96ab5c2ff612d9dfaae8c31f30c42168',
   'ff4f8391a6a40ca5b25d23bedd44a597'),
  ('6a118a874519e64e9963798a503f1d35',
   'dc43be40be0e53712f7e2bf5ca707209')
]

# 16-byte test key of all zeros.
test_key = bytes.fromhex('00000000000000000000000000000000')

aesCipher = Cipher(algorithms.AES(test_key),
          modes.ECB(),
```

```
          backend=default_backend())
aesEncryptor = aesCipher.encryptor()
aesDecryptor = aesCipher.decryptor()

# test each input
for index, kat in enumerate(nist_kats):
  plaintext, want_ciphertext = kat
  plaintext_bytes = bytes.fromhex(plaintext)
  ciphertext_bytes = aesEncryptor.update(plaintext_bytes)
  got_ciphertext = ciphertext_bytes.hex()

  result = "[PASS]" if got_ciphertext == want_ciphertext else "[FAIL]"

  print("Test {}. Expected {}, got {}. Result {}.".format(
    index, want_ciphertext, got_ciphertext, result))
```

BOOK PAGE 63

7.5 Line by line

Line(s)	What that line actually does
8 – 17	A list of (plaintext, expected ciphertext) pairs copied straight from the NIST file. Both are hex strings, not bytes the conversion happens later.
20	The all-zero test key. **`bytes.fromhex`** turns 32 hex characters into 16 bytes. Never use a key like this for anything real; it exists so the answers are reproducible worldwide.
22 – 26	The familiar Cipher/encryptor/decryptor construction from Listing 3-1.
29	`enumerate` gives us both the index and the pair, so the report can number the tests.
30 – 31	Unpacks the pair and converts the plaintext hex into the 16 raw bytes AES actually consumes.
32	One call to `update()` with exactly 16 bytes, so exactly one block comes back immediately. Because each vector is a full block, the buffering behaviour of Section 3 never gets a chance to confuse us here.
33	`.hex()` converts the ciphertext bytes back to a hex string so it can be compared with the expected value as text.
35	A conditional expression Python's one-line if/else. Evaluates to the string "[PASS]" or "[FAIL]".
37 – 38	Prints the verdict for each vector.

7.6 The output verified

Assuming that your processor is working correctly, you should see a 4/4 passing score. Here is the actual output produced when this listing was run for this tutorial:

$ python listing_3_2.py

```
Test 0. Expected 0336763e966d92595a567cc9ce537f5e, got 0336763e966d92595a567cc9ce537f5e.
Result [PASS].
Test 1. Expected a9a1631bf4996954ebc093957b234589, got a9a1631bf4996954ebc093957b234589.
Result [PASS].
Test 2. Expected ff4f8391a6a40ca5b25d23bedd44a597, got ff4f8391a6a40ca5b25d23bedd44a597.
Result [PASS].
Test 3. Expected dc43be40be0e53712f7e2bf5ca707209, got dc43be40be0e53712f7e2bf5ca707209.
Result [PASS].
```

WHY THIS MATTERS BEYOND THIS CHAPTER

A known-answer test is how every certified cryptographic module on earth proves it is not silently broken. Your phone runs these self-tests at boot. If a chip, a compiler optimisation or a porting bug corrupts the implementation, the KAT fails loudly instead of the system quietly producing garbage that looks encrypted.

Note the irony the authors are enjoying here: the only genuinely good use for ECB mode is proving that AES itself works.

EXERCISE 3.5 ALL NIST KATS

Write a program that will read one of these NIST KAT “rsp” files, and parse out the encryption and decryption KATs. Test and validate your AES library on all vectors on a couple of ECB test files.

BOOK PAGE 64

8. So What Is Wrong with ECB?

This all seems very reasonable. So what is wrong with ECB? Unless you have been completely asleep, you have noticed our dire warnings about it. Why? In a nutshell, because of its independence properties.

Let's return to Alice, Bob, and their nemesis Eve down in Antarctica. Alice and Bob are on a covert mission within the West Antarctica borders. They will send secret messages to each other over radio channels that Eve can monitor. Before they leave, they generate a shared key for encrypting and decrypting their messages, and they keep that key safe during their travels.

We can do that too. We will start by generating a key. Normally the key would be random, but we will just pick one that is easy to remember, so that we can also perfectly reproduce the following results. Here is the key:

```
1 key = bytes.fromhex('00112233445566778899AABBCCDDEEFF')
```

8.1 The EATSA form

Alice and Bob, being government agents, use a standardised EATSA form for sending each other messages. For example, to arrange a meeting:

the standard form

```
FROM: FIELD AGENT<codename>
TO: FIELD AGENT<codename>
RE: Meeting
DATE: <date>

Meet me today at <location> at <time>
```

If Alice is telling Bob to meet her at the docks at 11 p.m., the message would be:

Alice's actual message

```
FROM: FIELD AGENT ALICE
TO: FIELD AGENT BOB
RE: Meeting
DATE: 2001-1-1

Meet me today at the docks at 2300.
```

We will encrypt this message under the key we set out previously. We need to pad the message to make sure it is a multiple of 16 bytes long. We can do that by adding extra characters to the end until its length is a multiple of 16.

> **FOOTNOTE 5 Python's negative modulus**
>
> We take advantage of Python's convenient "negative modulus" behaviour, where -len(msg) % 16 is the same as 16 - (len(msg) % 16).

Check it yourself: if len(msg) is 108, then -108 % 16 evaluates to 4 in Python, which is exactly how many bytes of padding are needed. In C or Java the same expression gives -12, which is why this idiom surprises people arriving from those languages.

BOOK PAGE 65

8.2 Listing 3-3 AES ECB Padding

Listing 3-3. AES ECB Padding

```
1  # NEVER USE: ECB is not secure!
2  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
3  from cryptography.hazmat.backends import default_backend
4
5  # Alice and Bob's Shared Key
6  test_key = bytes.fromhex('00112233445566778899AABBCCDDEEFF')
7
8  aesCipher = Cipher(algorithms.AES(test_key),
9            modes.ECB(),
10           backend=default_backend())
11 aesEncryptor = aesCipher.encryptor()
12 aesDecryptor = aesCipher.decryptor()
13
14 message = b"""
15 FROM: FIELD AGENT ALICE
16 TO: FIELD AGENT BOB
17 RE: Meeting
18 DATE: 2001-1-1
19
20 Meet me today at the docks at 2300."""
21
22 message += b"E" * (-len(message) % 16)
23 ciphertext = aesEncryptor.update(message)
```

Line by line

Line(s)	What that line actually does
6	The fixed shared key, so that everyone reading the chapter gets identical ciphertext.
8 – 12	Identical to Listing 3-1 apart from the key.
14 – 20	A triple-quoted byte string. Note it begins immediately after the opening quotes, so the message starts with a newline character. **The exact whitespace matters** a single extra space anywhere changes every block from that point onward.
22	The padding. `-len(message) % 16` computes how many bytes are missing from a whole number of blocks, and that many “E” characters are appended.
23	One call to update() encrypts the whole padded message, because it is now an exact multiple of 16 bytes and nothing is left in the buffer.

This listing shows a straightforward but perhaps not optimal padding. We will use more standard approaches in the next section. It is, however, good enough for now. When Bob decodes his message, it will simply have a few extra “E” characters at the end.

> **WHY “E” IS A BAD PADDING SCHEME**

How does Bob know where the message ends and the padding begins? He does not. He has to guess that trailing E characters are padding but what if Alice's message genuinely ended in the letter E?

A correct padding scheme must be unambiguously removable. That is precisely what PKCS7, introduced in Section 17, provides, and precisely what SSLv3's scheme failed to do which is what makes the attack in Section 29 possible.

EXERCISE 3.6 SENDING BOB A MESSAGE

Using either a modification of the preceding program or your AES encryptor from the beginning of the chapter, create a couple of meetup messages from Alice to Bob. Also create a few from Bob to Alice. Make sure that you can correctly encrypt and decrypt the messages.

BOOK PAGE 66 – 67

9. Eve's View: Identical Blocks

With their new cryptographic technology at the ready, Alice and Bob begin surveillance in West Antarctica. They meet occasionally to share information and coordinate their activities. Meanwhile, Eve and her counter-intelligence colleagues learn of the infiltration and soon begin to identify the coded messages.

Take a look at several messages from Alice to Bob from Eve's perspective, where all she can see is the ciphertext. Do you notice anything? Consider these two messages:

message 1

```
FROM: FIELD AGENT ALICE
TO: FIELD AGENT BOB
RE: Meeting
DATE: 2001-1-1

Meet me today at the docks at 2300.
```

message 2

```
FROM: FIELD AGENT ALICE
TO: FIELD AGENT BOB
RE: Meeting
DATE: 2001-1-2

Meet me today at the town square at 1130.
```

Look at the two ciphertext outputs of these messages side by side. Note: even spacing and newlines matter, so make sure to use the format exactly as shown.

Block	Message 1 ciphertext	Message 2 ciphertext	Same?
1	0aee9b602c574044778d4f6de3481cb9	0aee9b602c574044778d4f6de3481cb9	IDENTICAL
2	0f3b683d1af60ed69396949eaf293eb2	0f3b683d1af60ed69396949eaf293eb2	IDENTICAL
3	44760fa0bb791339d557b43b250ac27c	44760fa0bb791339d557b43b250ac27c	IDENTICAL
4	8430e159229e4bf5c7b39fe1fb72cfab	8430e159229e4bf5c7b39fe1fb72cfab	IDENTICAL
5	a5c7412fda6ac67fe63093168f474913	c9b3ccefda71f286895b309d85245421	differ
6	dbd386db053613be242c6059539f93da	699f1cd5adbeb94b80980a0860ead320	differ
7	800d3ece3b12931be974f36ef5da4342	a8ff0ed2ca9b80908757f8c3ecbc9b0d	differ
8		9ff8e0d822c75122aa6df1fa6aa2be71	

A NOTE ON THESE HEX VALUES

These blocks were computed by running Listing 3-3 for this tutorial. Blocks 4 to 7 reproduce the book's printed values exactly; blocks 1 to 3 differ, because the first three blocks are extremely sensitive to the precise whitespace of the message and the book's printed values appear to come from a slightly different draft of the form.

Block	Message 1 ciphertext	Message 2 ciphertext	Same?

Do not chase the hex digits. The teaching point is the PATTERN, and it is identical either way: four of the blocks are the same across two different messages. Your own run will reproduce the table above if you type the message exactly as printed.

9.1 Why this happens

How many of the 16-byte blocks are identical? Four. Why? Remember that AES in its raw mode is like a code book. For every input and key, there is exactly one output, independent of any other inputs. Thus, because much of the message header is shared between messages, much of the output is also the same.

9.2 What Eve does with this

Eve and her colleagues notice the repeating elements of the messages they see day after day and soon start to figure out what they mean. How do they do this? They might make a good start by guessing. If you saw the same message being sent repeatedly, you could start to guess at some of its contents.

Another way to make progress might be to utilise a deserter or mole within the enemy organisation. They could conceivably get Eve a copy of the form or a discarded decoded message. All told, there are many ways for an adversary to learn about the structure and organisation of an encrypted message, and you should never assume otherwise. A common error made by those trying to protect information is to assume that the enemy cannot know some detail about how the system works.

9.3 Kerckhoff's principle

KERCKHOFF'S PRINCIPLE

Always live by Kerckhoff's principle. This nineteenth-century cryptographer long before modern computers taught that a cryptographic system must be secure even if everything is known about it, except the key.

That means we should find a way for our messages to be secure if the enemy knows just about everything about our system and merely lacks access to the key. It is the precise opposite of "security by obscurity", and it is why every algorithm in this book is public, published and openly attacked.

We made this silly example with an overly bureaucratic form, but even in real messages there is often a significant amount of predictable structure. Consider HTML, XML, or email messages. Those often have huge amounts of predictably positioned, identical data. It would be a terrible thing for an eavesdropper to start learning what is in a message just because it shares protocol headers with every other message.

9.4 The chosen-plaintext attack

Even worse, imagine if Eve's team can figure out a way to do what is called a chosen plaintext attack. In this attack, they figure out a way to get Alice or Bob to encrypt something on their behalf. Imagine, for example, that they figure out that Alice always calls a meeting with Bob after the Prime Minister of Western Antarctica

gives a public speech. Once they know this, they can use political speeches to trigger a message where much of the content is known. Or maybe they manage to slip Bob some false information to send to Alice, encrypted. Once they can control some or all of the plaintext, they can look at the encryption and begin to create their own code book.

BOOK PAGE 68

9.5 Splicing messages together

Eve can also easily create new messages by putting together bits and pieces of old messages. If Eve knows that the first blocks of a ciphertext are the header with a current date, she can take an old message body that directs Bob to an old meeting site and attach it to the new header. Then Bob ends up in the wrong place at the wrong time.

THIS IS AN INTEGRITY FAILURE, NOT A CONFIDENTIALITY FAILURE

Eve still cannot read a single word of either message. She does not have the key and she is not trying to get it. She is cutting and pasting sealed envelopes, and because ECB blocks are independent, the result still opens cleanly at Bob's end.

Notice how early in the chapter this appears. The authors introduce the integrity problem before they have even fixed the confidentiality problem, because the two are genuinely separate.

EXERCISE 3.7 SENDING BOB A FAKE MESSAGE

Take two different ciphertexts from Alice to Bob with different meeting instructions on different dates. Splice the ciphertext from the body of the first message into the body of the second message. That is, start by replacing the last block of the newer message with the last block (or blocks if it was longer) of the previous message. Does the message decrypt? Did you change where Bob goes to meet Alice?

10. The Image Experiment

All of this may still seem just a bit hypothetical. Perhaps ECB mode is not really all that bad. Perhaps it is only bad in extreme situations. Just in case there is a shadow of a doubt remaining, let's do one more test a pretty fun one to convince ourselves that ECB mode should never, ever be used for real message confidentiality.

In this experiment you will build a very basic AES encrypting program. It does not matter what key is used; feel free to generate a random one, or use a fixed test key. Read in a binary file, encrypt everything except the first 54 bytes, and then write it out to a new file.

10.1 Listing 3-4 AES Exercise Example

Listing 3-4. AES Exercise Example

```
# Partial Listing: Some Assembly Required

ifile, ofile = sys.argv[1:3]
with open(ifile, "rb") as reader:
  with open(ofile, "wb+") as writer:
    image_data = reader.read()
    header, body = image_data[:54], image_data[54:]
    body += b"\x00"*(16-(len(body)%16))
    writer.write(header + aesEncryptor.update(body))
```

Line by line

Line(s)	What that line actually does
1	The authors' standard marker: imports are omitted because nothing new is needed beyond the previous listings.
3	`sys.argv[1:3]` takes the two command-line arguments after the script name and unpacks them into the input and output filenames.
4 – 5	Nested `with` statements. Both files are opened in **binary** mode `"rb"` to read and `"wb+"` to write. Text mode here would corrupt the image.
6 – 7	Reads the whole file, then splits it into the first 54 bytes (the BMP header) and everything after. The header is left in the clear so the file is still a valid, viewable bitmap.
8	Zero-pads the body to a multiple of 16 so AES will accept it.
9	Writes the untouched header followed by the encrypted body.

FOOTNOTES 6 AND 7

This code listing does not show all the necessary imports, but it requires nothing new over previous listings. In the interest of space, the authors regularly leave out details shown in previous examples.

The reason we are not encrypting the first 54 bytes is because this program encrypts the contents of a bitmap file (BMP) and the header is 54 bytes in length. In real life, if the header were encrypted, you could just overwrite it with something reasonable based on the file size.

BOOK PAGE 69 – 70

10.2 The result actually run for this tutorial

Once you have this listing written, in the image editor of your choice create a large image with text that takes up most of the space. The book's image simply has the words TOP SECRET, at 800x600 pixels. Take your newly created file and run it through your encryption program, saving the output to something like encrypted_image.bmp. When finished, open the encrypted file in an image viewer. What do you see?



Figure 10.1 Book Figures 3-3 and 3-4, reproduced by actually running the program. Left: the original bitmap. Centre: the same file after real AES-ECB encryption the message is still perfectly legible. Right: the same file under AES-CTR, for comparison. Same algorithm, same key, same image; only the mode differs.

10.3 What happened here?

Why is the text of the image still so readable? AES is a block cipher that operates on 16 bytes at a time. In this image, many 16-byte chunks are the same. A chunk of black pixels is encoded with the same bits everywhere. Every time there is a 16-byte block of all black or all white, they encode to the same encrypted output. The structure of the image is thus still visible even once the individual 16-byte chunks are encrypted.

Really. Never use ECB. Leave that kind of thing to the "professionals" of the East Antarctica Truth-Spying Agency.

> **THE CENTRE IMAGE IS NOT AN ILLUSTRATION**
>
> That image is genuine AES-128 ciphertext. Every byte after the 54-byte header went through the real algorithm with a real key. AES is not broken here and the key was never leaked the mode simply preserved the structure of the plaintext, and structure was all an attacker needed.
>
> This single picture has probably prevented more ECB deployments than every written warning ever published. Put it in your lab record.

11. Wanted: Spontaneous Independence

To have an effective cipher, we need to:

- **Encrypt the same message differently each time.**
- **Eliminate predictable patterns between blocks.**

11.1 The first fix: the initialisation vector

To solve the first problem, we use a simple but effective trick to ensure that we never send the same plaintext twice, which means that we also never send the same ciphertext twice. We do this with an initialisation vector, or IV.

An IV is typically a random string that is used as a third input in addition to the key and plaintext into the encryption algorithm. Exactly how it is used depends on the mode, but the idea is to prevent a given plaintext from encrypting to a repeatable ciphertext.

Unlike the key, the IV is public. That is, one assumes that an attacker knows, or can obtain, the value of the IV. The presence of an IV does not help to keep things secret so much as it helps to keep them from being repeated, avoiding exposure of common patterns.

	Key	IV / nonce
Secret?	Yes absolutely	No assume the attacker has it
Purpose	makes the output unreadable	makes the output unrepeatable
Reused?	sometimes, carefully	NEVER with the same key
Size for AES	16, 24 or 32 bytes	always 16 bytes

11.2 The second fix: stop treating blocks as independent

As for the second problem, that of being able to eliminate patterns between blocks, we will solve it by introducing new ways to encrypt the message as a whole, rather than treating each block as an individual, independent mini-message like ECB mode does. The details of each solution are specific to the mode being used, but the principles generalise well.

BOOK PAGE 71 – 72

12. Not That Blockchain and a XOR Refresher

Recall from Chapter 2 that good hash algorithms are expected to have the avalanche property: a single change in one input bit will cause approximately half of the output bits to change. Block ciphers should have a similar property, and thankfully AES does. In ECB mode, however, the avalanche's impact is limited to the block size: if the plaintext is ten blocks long, a change in the very first bit will only change the output bits of the very first block. The remaining nine blocks will remain unchanged.

What if a change in the ciphertext of one block could affect all subsequent blocks? Well, it can, and it is quite easy to accomplish. When encrypting, for example, one can XOR the encrypted output of a block with the unencrypted input of the next block. To reverse this while decrypting, the ciphertext is decrypted and then the XOR operation is again applied to the previous ciphertext block to obtain the plaintext. This is called cipher block chaining (CBC) mode.

12.1 XOR, reviewed

Let's pause here for just a quick moment to review an operation called XOR, often written symbolically as ⊕. We are going to use XOR constantly throughout the book, so it is worthwhile to review it. XOR is a binary boolean operator with the following truth table, where we use 0 and 1 instead of "false" and "true".

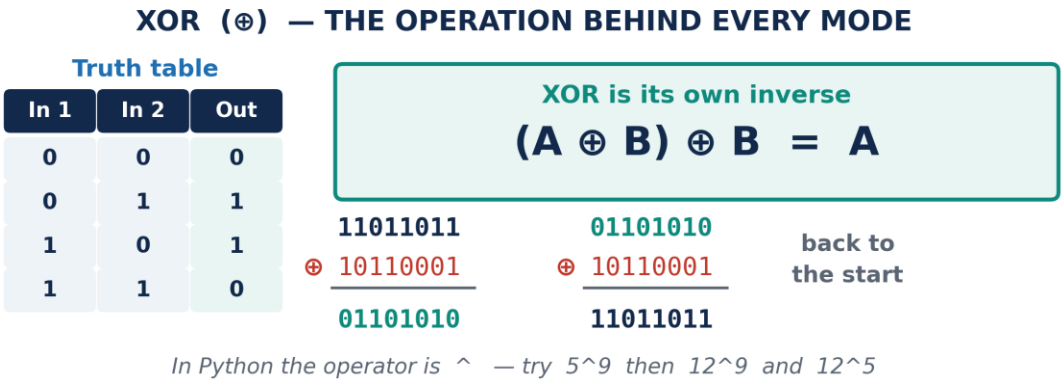


Figure 12.1 The XOR truth table, its inversion property, and a worked example on 8-bit values.

12.2 The inversion property

Truth tables are useful, showing precisely how functions like XOR behave for all combinations of inputs, but you actually do not need to think about XOR at this level. What is important is that XOR has an amazing inversion property: the XOR operation is its own inverse. That is, if you start with some binary number A and XOR it with B, you can recover A by XORing the output with B again.

$$(A \oplus B) \oplus B = A$$

Why does this work? If you look at Input 1 as a control bit, when it is 0, what comes out is simply Input 2. When Input 1 is 1, on the other hand, what comes out is the inverse of Input 2. If you take the outputs and

apply XOR with Input 1 again, it leaves the previously unchanged things unchanged (XOR with 0 again) while flipping the inverted things back to the way they were (XOR with 1 again).

Quite often we XOR not individual bits, but sequences of bits all at once. This is how we will use XOR throughout this book: as an operation between blocks of bits.

EXERCISE 3.8 XOR EXERCISE

Because we will use XOR so much, it is a good idea to get comfortable with XOR operations. In a Python interpreter, XOR a few numbers together. Python supports XOR directly using ^ as the operator. So, for example, 5^9 results in 12. What do you get when you try 12^9? What do you get when you try 12^5? Try this out with several different numbers.

the answers, verified

```
>>> 5 ^ 9
12
>>> 12 ^ 9
5     # the original first operand comes back
>>> 12 ^ 5
9     # the original second operand comes back
```

Both directions work because XOR is its own inverse, and because it is commutative: the two inputs are interchangeable.

EXERCISE 3.9 THE MASK OF XOR-O?

Although this exercise will be even more important in counter mode, it is useful to understand how XOR can be used to mask data. Create 16 bytes of plaintext (a 16-character message) and 16 bytes of random data (e.g., using os.urandom(16)). XOR these two messages together. There is no built-in operation for XORing a series of bytes, so you will have to XOR each byte individually using, for example, a loop. When you are done, take a look at the output. How “readable” is it? Now, XOR this output with the same random bytes again. What does the output look like now?

A helper you will reuse for the rest of the chapter

```
1  def xor_bytes(a, b):
2    return bytes(x ^ y for x, y in zip(a, b))
```

Line(s)	What that line actually does
1	Takes two byte strings of the same length.
2	`zip` walks both sequences in step, giving pairs of integers (iterating over a `bytes` object yields ints, not one-character strings). Each pair is XORed with `^`, and `bytes(...)` reassembles the result. Note that `zip` stops at the shorter input, which is exactly the behaviour you want when a partial block of key stream is left over.

BOOK PAGE 73 – 74

13. How CBC Mode Works

Returning from our XOR interruption to CBC: in this mode we XOR the output of one block of ciphertext with the next plaintext block. More precisely, if we call P[n] block n of plaintext and P'[n] block n of "munged, pre-encryption plaintext" (using the XOR operation to accomplish the very scientifically named munging process), we first create P'[n] from the previous encrypted block C[n−1], then we encrypt it to make C[n].

$$P'[n] = P[n] \oplus C[n-1]$$

From there we can apply AES encryption to P'[n], which is the length of an AES block, to get C[n]. When decrypting, then, we do not get the plaintext, we get the munged pre-encryption plaintext P'[n]. To get the actual plaintext, we need to reverse the preceding process, which we can do by running it through XOR with the previous encrypted block, recalling that XOR is its own inverse.

13.1 The algebra, step by step

Step	Expression	Justification
1	P'[n] = P[n] ⊕ C[n−1]	definition of the munged block
2	P'[n] ⊕ C[n−1] = P[n] ⊕ C[n−1] ⊕ C[n−1]	XOR both sides by C[n−1]
3	C[n−1] ⊕ C[n−1] = 0	anything XORed with itself is zero
4	P'[n] ⊕ C[n−1] = P[n]	and X ⊕ 0 = X

Thus, to get the original plaintext when decrypting, we need only XOR the decrypted block with the previous encrypted block. The very first block, which has no predecessor, is simply XORed with the initialisation vector after decryption. That is the essence of CBC mode: every block is dependent on the blocks that came before.

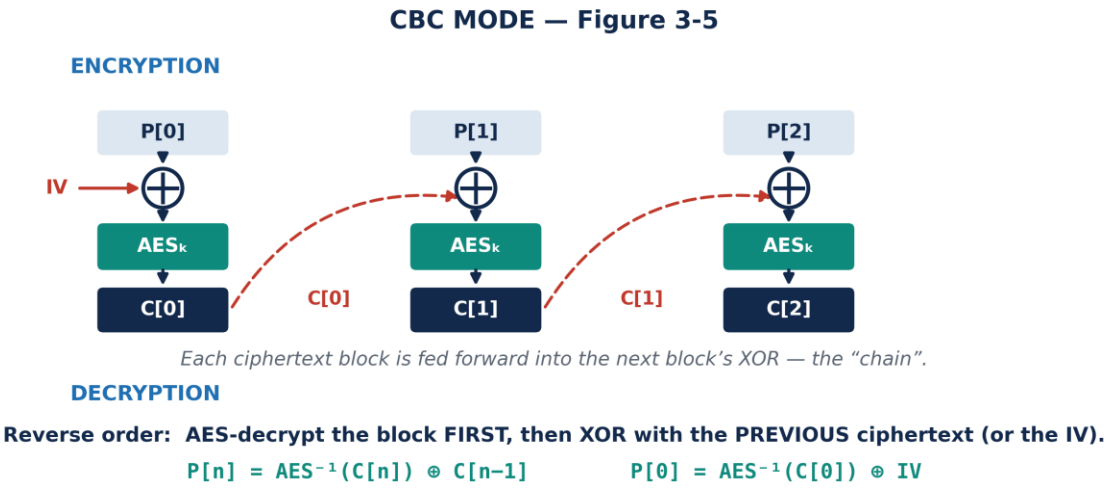


Figure 13.1 Visual depictions of CBC encryption and decryption. Note that in encryption, the first block of plaintext is XORed with the IV before AES, while in decryption, the ciphertext goes through AES first and is then XORed with the IV to correctly reverse the encryption process. (Book Figure 3-5.)

THE ORDER IS REVERSED THIS IS THE COMMONEST CBC MISTAKE

Encrypting: XOR first, THEN AES. Decrypting: AES first, THEN XOR.

Students who implement Exercise 3.12 and get garbage out have almost always XORed before decrypting instead of after. Look at the figure again and trace one block through both directions before you write any code.

13.2 What the chain buys us

In CBC mode, changes to any input block thus affect the output block for all subsequent blocks. This does not produce a complete or perfect avalanche property because it does not affect any preceding blocks, but even having the avalanche effect moving forward prevents exposing the kinds of patterns that we observe in ECB mode.

13.3 Configuring CBC

Configuration of CBC mode is mostly familiar: we generate a key and then take the extra step of generating an initialisation vector. Because the IV is XORed with the first block, AES-CBC IVs are always 128 bits long (16 bytes), even if the key size is larger (typically 192 or 256 bits). In the following example, the key is 256 bits and the IV is 128 bits, as it must be.

FOOTNOTE 8

“We promise to use more initialisms next time.”

BOOK PAGE 75

13.4 Listing 3-5 AES-CBC

Listing 3-5. AES-CBC

```
1  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
2  from cryptography.hazmat.backends import default_backend
3  import os
4
5  key = os.urandom(32)
6  iv = os.urandom(16)
7
8  aesCipher = Cipher(algorithms.AES(key),
9            modes.CBC(iv),
10           backend=default_backend())
11 aesEncryptor = aesCipher.encryptor()
12 aesDecryptor = aesCipher.decryptor()
```

Line(s)	What that line actually does
5	32 bytes = 256 bits. This is AES-256. Compare Listing 3-1, which used 16 bytes for AES-128 the only difference between the variants is the length of this one value.
6	16 bytes, always, regardless of the key length. The IV must be the **block** size because it is XORed with a block.
9	The only structural change from Listing 3-1: `modes.ECB()` becomes `modes.CBC(iv)`. Notice that `algorithms.AES` takes the key while `modes.CBC` takes the IV; AES always needs a key, but the use of an IV is dependent on the mode.
11 – 12	Unchanged. The whole point of the API design is that the rest of your program does not care which mode you picked.

14. Proper Padding

While we are in the business of improving things, let's introduce a better padding mechanism. The cryptography module provides two schemes, one following what is known as the PKCS7 specification and the other following ANSI X.923.

Scheme	Rule	3 bytes of padding	2 bytes of padding
PKCS7	append n bytes, each holding the value n	\x03\x03\x03	\x02\x02
ANSI X.923	append zeros, last byte holds the total length	\x00\x00\x03	\x00\x02

The cryptography module provides a padding context that is analogous to the AES cipher context. In the next code listing, padder and unpadder objects are created for adding and removing padding. Note that these objects also use update and finalize, since no padding is created from calling the update() method. It does, however, return full blocks, storing the rest of the bytes for either the next call to update() or the finalize() operation. When finalize() is called, all remaining bytes are returned along with enough bytes of padding to make a full block size.

Although the API seems straightforward, it does not necessarily behave as one might expect.

WHY PKCS7 IS UNAMBIGUOUS AND "EEEE" WAS NOT

Under PKCS7 the receiver reads the very last byte, sees the number n, and removes exactly n bytes. There is never any doubt, because the padding declares its own length.

Crucially, padding is added even when the message is already an exact multiple of the block size a whole extra block of \x10 bytes. Without that rule a message ending in the byte \x01 would be indistinguishable from one byte of padding.

BOOK PAGE 76

14.1 Listing 3-6 AES-CBC Padding

Listing 3-6. AES-CBC Padding

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import padding
import os

key = os.urandom(32)
iv = os.urandom(16)

aesCipher = Cipher(algorithms.AES(key),
          modes.CBC(iv),
          backend=default_backend())
aesEncryptor = aesCipher.encryptor()
aesDecryptor = aesCipher.decryptor()

# Make a padder/unpadder pair for 128 bit block sizes.
padder = padding.PKCS7(128).padder()
unpadder = padding.PKCS7(128).unpadder()

plaintexts = [
  b"SHORT",
  b"MEDIUM MEDIUM MEDIUM",
  b"LONG LONG LONG LONG LONG LONG",
]

ciphertexts = []

for m in plaintexts:
  padded_message = padder.update(m)
  ciphertexts.append(aesEncryptor.update(padded_message))

ciphertexts.append(aesEncryptor.update(padder.finalize()))

for c in ciphertexts:
  padded_message = aesDecryptor.update(c)
  print("recovered", unpadder.update(padded_message))

print("recovered", unpadder.finalize())
```

14.2 The output run and verified

Run the code in Listing 3-6 and observe the output. Is it what you expected? It should have looked like this:

```
$ python listing_3_6.py
recovered b''
recovered b''
recovered b'SHORTMEDIUM MEDIUM MEDIUMLONG LO'
recovered b'NG LONG LONG LON'
recovered b'G LONG'
```

Why did it not produce the original messages exactly as specified? Three messages went in and five outputs came out, two of them empty.

14.3 Semantics matter

There is technically nothing incorrect with this code, but there is definitely a mismatch between the apparent intention of the code and the actual output. This code suggests that the author intended to encrypt each one of the three strings as an independent message. That is not what we got.

Let's talk about the update() and finalize() API one more time. Because of how these methods behave for certain modes, it can be tempting to think about update() as a stand-alone encryptor wherein a plaintext block is provided as input and a ciphertext block is provided as output.

In reality, the API is designed such that the number of calls to update() is irrelevant. What is being encrypted is not the input to update(), but the concatenation of every input to some number of update() calls, plus of course any output from a finalize() call at the end. Thus, the program in Listing 3-6 is not encrypting three inputs and producing five outputs; it is processing a single continuous input and producing a single continuous output.

BOOK PAGE 78 – 79

14.4 Padding never appears until finalize()

Understanding the update() and finalize() API is especially important for the padding operations that we have introduced. Padding behaviour can appear unusual if you try to think of update() as an independent operation. Note that individual calls to update() produce no padding. Only the finalize() operation will do that.

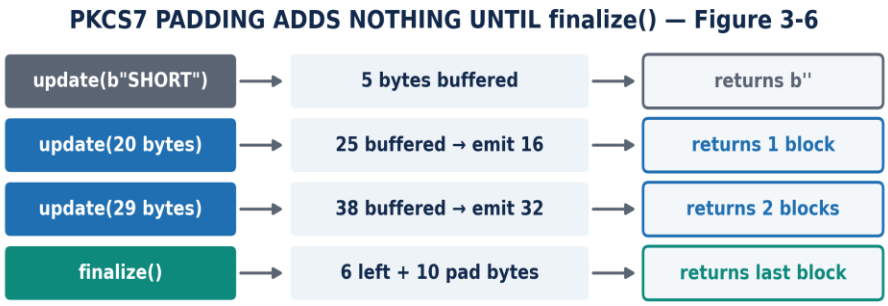


Figure 14.1 PKCS7 padding does not add any padding until the finalize operation. (Book Figure 3-6.)

Unpadding can be even more jarring. Unlike the padding operations, you can submit a full block to the unpadder and still get nothing back. This is because the unpadder has to reserve the last block received in update() calls in case it is the last block. Because unpadding requires examining the last block, the unpadder has to be sure it has received all the blocks to know that it has the last one.

14.5 The complete walkthrough

Walking through Listing 3-6 one more time illustrates how the effects of these operations are compounded when the padder and encryptor are used together.

Pass	Input	Padder buffer	Padder returns	Encryptor returns
1	SHORT (5 bytes)	5	b''	b'' nothing to encrypt
2	MEDIUM… (20 bytes)	5 + 20 = 25 → keep 9	first 16 bytes	one block
3	LONG… (29 bytes)	9 + 29 = 38 → keep 6	two full blocks	two blocks
finalize		6 + 10 pad bytes	one final block	one block

Once the loop exits, the padder's finalize method is called. It takes the last bytes of input, appends the necessary padding, and passes it to the encryption operation. The ciphertext is appended to the list and encryption is over. There are now four ciphertext messages to decrypt.

Reversing the process, the first message is, you may recall, the empty buffer. It just passes straight through everything and comes out as an empty message. But the next recovered text is also empty. That is because the first full block to the unpadder is reserved for the reasons we explained. It produces an empty output fed

into the AES decryptor's update() method. This generates our second empty output. The remaining three are more straightforward.

BOOK PAGE 80

14.6 The principle

This was intentional. The crucial principle is that semantics matter. How we think about our code can be different from how it operates, and this can result in unexpected, and often insecure, results. When you use a library always better than creating your own you must understand the API's approach and design. It is critical that you think the way the API is designed to be used.

> **THE RULE FOR THE cryptography LIBRARY**
>
> Always think about everything submitted to a sequence of encryption update() calls and one finalize() call as a SINGLE input.
>
> Similarly, think about everything recovered from a series of decryption update() calls and one finalize() call as a SINGLE output.

15. Listing 3-7 The Broken Manager

Let's look at another way to do this wrong. Suppose we decide to create our own API that will actually work on a message level: every message can be encrypted and decrypted individually and independently.

> **FOOTNOTE 9**
>
> "Yes, this is a theme in the book. We have found that it is often when things are broken that they are best understood."

Listing 3-7. Broken AES-CBC Manager

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import padding
import os

class EncryptionManager:
  def __init__(self):
    self.key = os.urandom(32)
    self.iv = os.urandom(16)

  def encrypt_message(self, message):
    # WARNING: This code is not secure!!
    encryptor = Cipher(algorithms.AES(self.key),
              modes.CBC(self.iv),
              backend=default_backend()).encryptor()
    padder = padding.PKCS7(128).padder()

    padded_message = padder.update(message)
    padded_message += padder.finalize()
    ciphertext = encryptor.update(padded_message)
    ciphertext += encryptor.finalize()
    return ciphertext
```

```
23
24    def decrypt_message(self, ciphertext):
25      # WARNING: This code is not secure!!
26      decryptor = Cipher(algorithms.AES(self.key),
27                modes.CBC(self.iv),
28                backend=default_backend()).decryptor()
29      unpadder = padding.PKCS7(128).unpadder()
30
31      padded_message = decryptor.update(ciphertext)
32      padded_message += decryptor.finalize()
33      message = unpadder.update(padded_message)
34      message += unpadder.finalize()
35      return message
36
37  # Automatically generate key/IV for encryption.
38  manager = EncryptionManager()
39
40  plaintexts = [
41    b"SHORT",
42    b"MEDIUM MEDIUM MEDIUM",
43    b"LONG LONG LONG LONG LONG LONG"
44  ]
45
46  ciphertexts = []
47
48  for m in plaintexts:
49    ciphertexts.append(manager.encrypt_message(m))
50
51  for c in ciphertexts:
52    print("Recovered", manager.decrypt_message(c))
```

Run the code and observe the output. Did you get each message individually this time? Good! You probably like this version a lot better.

The API might be more semantically aligned this time, but the implementation is very broken and incredibly dangerous. Before we tell you what is wrong with it, can you try and see it yourself? Are there any security principles we have talked about in this chapter that we are violating?

BOOK PAGE 82

16. A Key to Hygienic IVs

The problem with Listing 3-7 is that it is reusing the same key and IV for different messages. Take a look at the constructor where the key and IV are created. Using that single key/IV pair, the offending code re-creates encryptor and decryptor objects in every call to encrypt_message and decrypt_message. Remember, the IV is supposed to be different each time you encrypt, preventing the same data from being encrypted to the same ciphertext. This is not optional.

16.1 See the failure for yourself

encrypting the same plaintext twice with Listing 3-7

```
>>> b = EncryptionManager()
>>> b.encrypt_message(b"SHORT").hex()
'2defbe555099bd0967ba1f0694a4e43b'
>>> b.encrypt_message(b"SHORT").hex()
'2defbe555099bd0967ba1f0694a4e43b'
```

Identical input, identical output exactly the ECB disease we spent ten pages curing, reintroduced through the back door by a well-meaning API design.

16.2 Why update() is different from a new encryptor

Once again, it is important to understand how an API is built and the security parameters associated with it. Remember that in CBC encryption, the algorithm combines the first plaintext block with the IV using the XOR operation before the AES operation is applied. Each subsequent plaintext block is combined with the previous ciphertext block using XOR before AES encryption. With the Python API, each call to update() adds blocks to this chain, leaving data less than a full block in an internal buffer for subsequent calls. The finalize() method does not actually do any more encrypting, but will raise an error if there is incomplete data still waiting to be encrypted.

Calling the update() method over and over is not reusing a key and IV, because we are appending to the end of the CBC chain. On the other hand, if you create new encryptor and decryptor objects, as we did in Listing 3-7, you are re-creating the chain from the beginning. If you reuse a key and IV here, you get exactly the same output for the same input every time.

> **THE RULE**
>
> When using the API of Python's cryptography module, never give the same key and IV pair to an encryptor more than once. Obviously, you give the same key and IV to the corresponding decryptor.
>
> In fact, it is probably best to never reuse the same key again, period.

BOOK PAGE 83

17. Listing 3-8 The Corrected Manager

Here we correct our previous error and only use a key/IV pair once. The encryptor and decryptor objects are moved to the constructor and, instead of having a single encrypt_message() or decrypt_message() call, we use the update/finalize pattern used by the cryptography module.

Listing 3-8. AES-CBC Manager

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import padding
import os

class EncryptionManager:
  def __init__(self):
    key = os.urandom(32)
    iv = os.urandom(16)
    aesContext = Cipher(algorithms.AES(key),
              modes.CBC(iv),
              backend=default_backend())
    self.encryptor = aesContext.encryptor()
    self.decryptor = aesContext.decryptor()
    self.padder = padding.PKCS7(128).padder()
    self.unpadder = padding.PKCS7(128).unpadder()

  def update_encryptor(self, plaintext):
    return self.encryptor.update(self.padder.update(plaintext))

  def finalize_encryptor(self):
    return (self.encryptor.update(self.padder.finalize())
        + self.encryptor.finalize())

  def update_decryptor(self, ciphertext):
    return self.unpadder.update(self.decryptor.update(ciphertext))

  def finalize_decryptor(self):
    return (self.unpadder.update(self.decryptor.finalize())
        + self.unpadder.finalize())

# Auto generate key/IV for encryption
manager = EncryptionManager()

plaintexts = [
  b"SHORT",
  b"MEDIUM MEDIUM MEDIUM",
  b"LONG LONG LONG LONG LONG LONG"
]

ciphertexts = []

for m in plaintexts:
```

```
  ciphertexts.append(manager.update_encryptor(m))
ciphertexts.append(manager.finalize_encryptor())

for c in ciphertexts:
  print("Recovered", manager.update_decryptor(c))
print("Recovered", manager.finalize_decryptor())
```

17.1 Line by line

Line(s)	What that line actually does
8 – 9	Note that these are now **local** variables, not `self.key` and `self.iv`. They are used once to build the context and then deliberately thrown away, so no method can accidentally reuse them.
10 – 12	One single context is created, once, in the constructor. This is the entire fix.
13 – 16	Encryptor, decryptor, padder and unpadder are all stored as instance state and live for the object's whole lifetime.
18 – 19	Padding and encryption composed in one expression, read inside out: pad first, then encrypt. Each call appends to the same CBC chain.
21 – 23	Two things must happen at the end, in this order: push the padder's final bytes through the encryptor, then close the encryptor. Concatenating both results is what returns the complete tail of the ciphertext.
25 – 26	The mirror image: decrypt, then unpad.
28 – 30	And the mirror of finalisation. Note `self.decryptor.finalize()` is passed through the unpadder's `update`, not its finalize the unpadder is closed separately on the next line.

17.2 The verified output

$ python listing_3_8.py

```
Recovered b''
Recovered b''
Recovered b'SHORTMEDIUM MEDIUM MEDIUMLONG LO'
Recovered b'NG LONG LONG LON'
Recovered b'G LONG'
```

This listing does not reuse key/IV pairs, but you have probably noticed that we are no longer treating the individual messages as individual messages. Now that we are back to the update()/finalize() pattern, we have to treat all the data passed to a single context as a single input. If we want each message treated separately, we need a sequence of update() calls and a finalize() call per input. Alternatively, we can submit all three messages as a single input from the perspective of the encryption and decryption and have an independent mechanism for splitting the single decryption output into messages.

> **THE TRADE-OFF, STATED PLAINLY**
>
> Listing 3-7 has the API you want and is catastrophically insecure. Listing 3-8 is secure and has an API that does not match how you think about messages.

> Neither is wrong; they are answering different questions. There is nothing wrong with a per-message API so long as a key and IV are not reused which means generating a fresh IV for every message and transmitting it alongside the ciphertext. That is what real protocols do.

In summary, it is important to carefully understand any cryptography APIs that you use, how they work, and what their requirements especially security requirements are. It is also important to understand how easy it can be to create an API that looks like it does the right thing but is actually leaving you vulnerable.

Remember, YANAC You Are Not A Cryptographer… yet! Do not roll your own crypto like we are doing in these educational examples.

BOOK PAGE 85

17.3 Why update/finalize exists at all

So why does the cryptography module use the update/finalize pattern? Quite often, data needs to be processed in chunks in many practical cryptographic operations. Suppose that you are transmitting data over the network. Do you really want to wait until you have the entire content before you can encrypt it? Even if you were encrypting a local file on the hard drive, it might be impractically large for all-at-once encryption. The update() method allows you to feed data to an encryption engine as it becomes available.

The finalize() operation is useful for enforcing requirements such as that the CBC operation did not leave an incomplete block unencrypted, and that the session is over.

EXERCISE 3.10 DETERMINISTIC OUTPUT

Run the same inputs through AES-CBC using the same key and IV. You can use Listing 3-7 as a starting point. Change the inputs to be the same each time and print out the corresponding ciphertexts. What do you notice?

EXERCISE 3.11 ENCRYPTING AN IMAGE

Encrypt the image that you encrypted with ECB mode earlier. What does the encrypted image look like now? Don't forget to leave the first 54 bytes untouched!

EXERCISE 3.12 HAND-CRAFTED CBC

ECB mode is just raw AES. You can create your own CBC mode using ECB as the building block. For this exercise, see if you can build a CBC encryption and decryption operation that is compatible with the cryptography library. For encryption, remember to take the output of each block and XOR it with the plaintext of the next block before encryption. Reverse the process for decryption.

Footnote 10: Never use this for production code! Always use well-tested libraries.

A skeleton for Exercise 3.12

Build CBC out of raw ECB blocks

```
def cbc_encrypt(key, iv, plaintext):   # plaintext must be a multiple of 16
  ecb = Cipher(algorithms.AES(key), modes.ECB(),
         backend=default_backend()).encryptor()
  previous = iv
  out = b""
  for i in range(0, len(plaintext), 16):
    block = plaintext[i:i+16]
    munged = xor_bytes(block, previous)  # XOR first
    cblock = ecb.update(munged)      # then AES
    out += cblock
    previous = cblock           # chain it forward
  return out
```

Line(s)	What that line actually does
4	The chain starts at the IV. After the first block it becomes the previous ciphertext block.
8 – 9	The two operations in the correct order for ENCRYPTION: XOR, then AES. For decryption you must swap them AES-decrypt, then XOR.
11	The line that makes it a chain. Forget it and you have written ECB with extra steps.

BOOK PAGE 86

18. Cross the Streams Counter Mode

Counter mode has a number of advantages over CBC mode and, in the authors' opinion, is significantly easier to understand than CBC mode. Also, while CTR is the traditional abbreviation, "CM" is a really nice set of initials.

Although simple, the concept behind this mode can be a little counter-intuitive at first. In CTR mode, you actually never use AES for encryption or decryption of the data. Instead, this mode generates a key stream that is the same length as the plaintext and then uses XOR to combine them together.

18.1 The one-time pad connection

Recall from earlier exercises in this chapter that XOR can be used to mask plaintext data by combining it with random data. Exercise 3.9 masked 16 bytes of plaintext with 16 bytes of random data. This is a real form of encryption called a one-time pad (OTP). It works great but requires that the key is the same size as the plaintext. We do not have the space here to explore the OTP further; the important concept is that using XOR to combine plaintext and random data is a great way to create ciphertext.

AES-CTR mimics this aspect of the OTP. But instead of requiring the key to be the same size as the plaintext a real pain when encrypting a 1TB file it uses AES and a counter to generate a key stream of almost arbitrary length from an AES key as small as 128 bits.

18.2 How the key stream is built

To do this, CTR mode uses AES to encrypt a 16-byte counter, which generates 16 bytes of key stream. To get 16 more bytes of key stream, the mode increases the counter by one and encrypts the updated 16 bytes. By continually increasing the counter and encrypting the result, CTR mode can produce an almost arbitrary amount of key stream material. Once a sufficient amount of key material is generated, the XOR operation is used to combine them together to produce the ciphertext.

Although the counter is changing by a small amount each time often just changing by a single bit AES has good per-block avalanche properties. Thus, each output block appears completely different from the last, and the stream as a whole appears to be random data.

> **FOOTNOTE 11**
>
> There are limits to how much key stream a single key may safely produce, but these are beyond the scope of this book.

> **NOTE: RANDOM THOUGHTS**
>
> Randomness is actually a huge deal in cryptography. Many otherwise acceptable algorithms have been compromised in practice if they did not have sufficient sources of randomness for keys, among other things. The OTP algorithm we briefly mentioned requires a key that is the same size as the plaintext, no

matter how large, and that the entire key be truly random data. AES-CTR mode only requires that the AES key be truly random.

The key stream produced by AES-CTR looks random, but is actually pseudo-random. This means that if you know the AES key, you know the whole key stream no matter how random it appears to be.

Ensuring that you have a sufficiently random source of data is beyond the scope of this book. For our purposes, we will assume that os.urandom() can return acceptably random data for our needs. In production cryptography environments, you would need to analyse this far more carefully. Randomness is so important that we will mention it more than once in fact, we return to it near the end of this very chapter.

BOOK PAGE 87 – 88

18.3 The formulas

Although AES-CTR is a stream cipher, we can still think about it one block at a time. To encrypt any given block of plaintext, generate the key stream for that block's index and XOR it with the (possibly partial) block.

$$C[n] = P[n] \oplus AES_k(IV + n)$$

The only other slight twist is that we do not want to start with the same counter value every time. So our IV, which we will call our nonce, is used as the starting counter value.

XOR is a really versatile mathematical operation. You can think of it as controlled bit-flipping: to compute A ⊕ B, you march down their bits in tandem; when you encounter a 1 in B, you invert the corresponding bit in A, and when you encounter a 0 in B, you leave that bit in A alone. Thinking of it that way, it is easy to see how doing that twice simply restores A to what it was before.

More formally, as discussed earlier, XOR is its own inverse. Since we created a stream of encrypted blocks by applying XOR to the appropriate value in the key stream, we simply do exactly the same thing to decrypt:

$$P[n] = C[n] \oplus AES_k(IV + n)$$

Of course, nothing happens if you merely XOR with 0, since A ⊕ 0 = A, which is where the inverse property comes from. So the keys in the stream need to be composed of random-looking bits but that is exactly the type of key stream that AES produces.

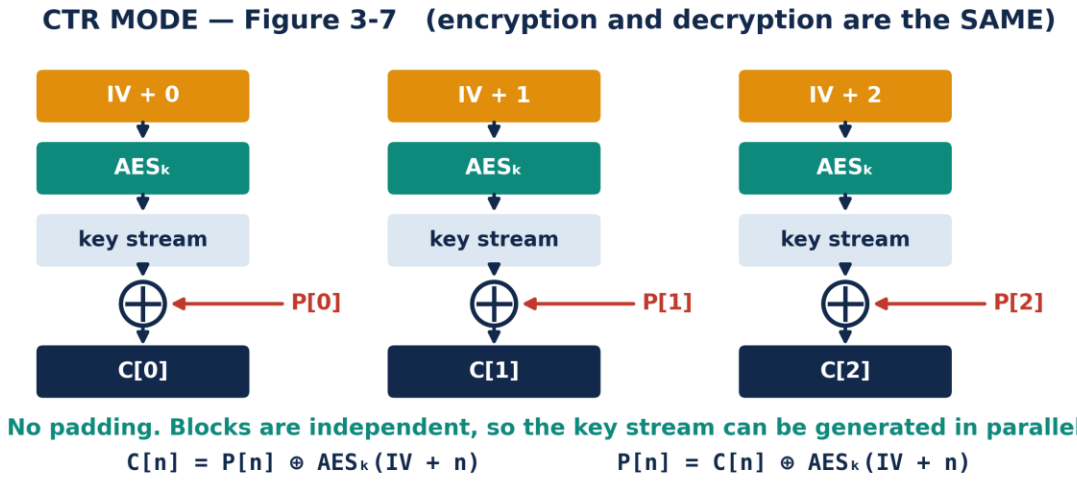


Figure 18.1 Visual depictions of CTR encryption and decryption. Note that encryption and decryption are the same process. (Book Figure 3-7.)

READ THOSE TWO FORMULAS AGAIN

They are the same formula. Encryption and decryption in counter mode are literally the identical operation which is why the CTR figure needs only one diagram where CBC needed two.

There is a second consequence that matters enormously: AES is only ever run in the ENCRYPT direction, even when you are decrypting. The AES decryption circuit is never used in CTR mode at all.

Happily, stream ciphers do not require padding. It is quite simple to only XOR a partial block, discarding the later parts of the key that are not needed.

BOOK PAGE 89

18.4 Listing 3-9 AES-CTR

In general, this approach is much simpler. Padding goes away and blocks can again be encrypted independent of one another. Let's see it in action in the cryptography module.

Listing 3-9. AES-CTR

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
import os

class EncryptionManager:
  def __init__(self):
    key = os.urandom(32)
    nonce = os.urandom(16)
    aes_context = Cipher(algorithms.AES(key),
                modes.CTR(nonce),
                backend=default_backend())
    self.encryptor = aes_context.encryptor()
    self.decryptor = aes_context.decryptor()

  def updateEncryptor(self, plaintext):
    return self.encryptor.update(plaintext)

  def finalizeEncryptor(self):
    return self.encryptor.finalize()

  def updateDecryptor(self, ciphertext):
    return self.decryptor.update(ciphertext)

  def finalizeDecryptor(self):
    return self.decryptor.finalize()

# Auto generate key/IV for encryption
manager = EncryptionManager()

plaintexts = [
  b"SHORT",
  b"MEDIUM MEDIUM MEDIUM",
  b"LONG LONG LONG LONG LONG LONG"
]

ciphertexts = []

for m in plaintexts:
  ciphertexts.append(manager.updateEncryptor(m))
ciphertexts.append(manager.finalizeEncryptor())

for c in ciphertexts:
  print("Recovered", manager.updateDecryptor(c))
print("Recovered", manager.finalizeDecryptor())
```

18.5 Line by line

Line(s)	What that line actually does
8	Called a **nonce** rather than an IV, because in counter mode it is the starting value of the counter. It is still 16 bytes, still public, and still must never repeat under the same key.
10	The single change from Listing 3-8: `modes.CBC(iv)` becomes `modes.CTR(nonce)`.
15 – 16	Notice what is **missing**: there is no padder anywhere in this class. Stream ciphers do not need padding, so half the complexity of Listing 3-8 simply evaporates.
18 – 19	Because no padding is needed, the finalize methods are actually unnecessary except for "closing" the object. They are kept for symmetry and pedagogy.
21 – 25	Decryption is the same two methods again. The object does not distinguish encryption from decryption because, mathematically, they are the same operation.

18.6 The verified output compare this with Listing 3-8

$ python listing_3_9.py

```
Recovered b'SHORT'
Recovered b'MEDIUM MEDIUM MEDIUM'
Recovered b'LONG LONG LONG LONG LONG LONG'
Recovered b''
Recovered b''
```

LOOK AT WHAT JUST HAPPENED

Each message came back individually and intact the very behaviour Listing 3-6 and Listing 3-8 could not deliver. Nothing clever was done to achieve it: with no padder in the chain, there is no buffering, so every byte fed to update() comes straight back out.

This is the clearest possible demonstration of why the block/stream distinction of Section 5 actually matters in practice.

BOOK PAGE 90 – 91

18.7 Choosing between CTR and CBC

How do you choose between CTR and CBC modes? In almost all circumstances, counter mode is recommended. Not only is it easier, but in some circumstances it is also more secure. As if that was not enough, counter mode is also easier to parallelise, because keys in the key stream are computed from their index, not from a preceding computation.

FOOTNOTE 12

You can remember it because CTR also stands for “choose the right”. You can even buy “CTR” rings as a friendly, constant and, when twisted a bit to our purposes, cryptographic reminder.

Why even talk about CBC, then? At the very least it is still in wide use, so you will benefit from understanding it when you encounter it in the wild. We will introduce other modes later in the book that build on counter mode to make something even better. For now, it is enough to understand the basic characteristics of CBC and CTR modes and how each one works to build a better algorithm from an underlying block cipher.

	CBC	CTR
Cipher type	block	stream
Padding	required	none
Encrypt block n needs	ciphertext of block n−1	only the nonce and the index n
Parallelisable?	no strictly sequential	yes fully
AES direction used	encrypt and decrypt	encrypt only, in both directions
Effect of one flipped ciphertext byte	wrecks this block and shifts all later ones	flips exactly one plaintext byte
Recommended?	legacy; understand it	yes, in almost all circumstances

EXERCISE 3.13 WRITE A SIMPLE COUNTER MODE

As you did with CBC, create counter mode encryption from ECB mode. This should be even easier than it was with CBC. Generate the key stream by taking the IV block and encrypting it, then increasing the value of the IV block by one to generate the next block of key stream material. When finished, XOR the key stream with the plaintext. Decrypt in the same manner.

EXERCISE 3.14 PARALLEL COUNTER MODE

Extend your counter mode implementation to use a thread pool to generate the key stream in parallel. Remember that to generate a block of key stream, all that is required is the starting IV and which block of key stream is being generated (e.g. 0 for the first 16-byte block, 1 for the second, and so on). Start by creating a function that can generate any particular block of key stream, perhaps something like

keystream(IV, i). Next, parallelise the generation of a key stream up to n by dividing the counter sequence among independent processes any way you please, and have them all work on generating their key stream blocks independently.

A skeleton for Exercises 3.13 and 3.14

Counter mode built from raw ECB

```
1  def keystream_block(key, iv, i):
2    counter = (int.from_bytes(iv, "big") + i) % (1 << 128)
3    block = counter.to_bytes(16, "big")
4    ecb = Cipher(algorithms.AES(key), modes.ECB(),
5           backend=default_backend()).encryptor()
6    return ecb.update(block)
7
8
9  def ctr_crypt(key, iv, data):     # encrypt AND decrypt
10   out = b""
11   for i in range(0, len(data), 16):
12     chunk = data[i:i+16]
13     ks = keystream_block(key, iv, i // 16)
14     out += xor_bytes(chunk, ks)  # zip stops at the shorter one
15   return out
```

Line(s)	What that line actually does
2	Treats the 16-byte nonce as one enormous integer and adds the block index. The modulus makes the counter wrap around rather than overflow past 128 bits.
3	Converts it back to exactly 16 bytes, big-endian, ready to feed to AES.
6	This is the whole of “encryption” in counter mode AES is applied to the COUNTER, never to your data.
9	One function for both directions, because they are the same operation. Call it twice and you get your original data back.
14	The partial final block needs no special handling: `zip` inside `xor_bytes` stops at the shorter sequence and the unused key stream is simply discarded.
13	For Exercise 3.14, note that this line depends only on `i`. Every block can therefore be computed by a different thread or process at the same time which is exactly what CBC forbids.

BOOK PAGE 91 – 92

19. Key and IV Management

As you have seen, having a library such as cryptography makes all kinds of encryption convenient and simple to use. Unfortunately, this simplicity can be deceptive and lead to mistakes; there are many ways to get it wrong. We have already touched briefly on one of them: reuse of keys or IVs. That kind of mistake falls under the broader category of key and IV management, and doing it incorrectly is a common source of problems.

> **IMPORTANT**
>
> You must never reuse key and IV pairs. Doing so seriously compromises security and disappoints cryptography book authors. Just don't do it. Always use a new key/IV pair when encrypting anything.

19.1 Why reuse hurts in CBC mode

Why don't you want to reuse a key and IV pair? For CBC, we already mentioned one of the potential problems: if you reuse a key and IV pair, you will get predictable output for predictable headers. Parts of your messages that you might be inclined not to think about at all, because they are boilerplate or contain hidden structure, will become a liability; adversaries can use predictable ciphertext to learn about your keys.

Think about an HTML page, for example. The first characters are often the same across multiple pages `"<!DOCTYPE html>\n"`. If the first 16 bytes (an AES block) of HTML pages are the same and you encrypt them under the same key/IV pair, the ciphertext will be the same for each one. You have just leaked data to your enemy, and they can start to analyse your encrypted data for patterns.

If your web site has a large amount of static content or dynamic results that are identically generated, each encrypted page becomes uniquely identifiable. The enemy may not know what each page says, but they can determine the frequency of use and track which parties receive the same pages.

Reusing a key and IV in CBC mode is bad.

19.2 Why reuse is catastrophic in counter mode

Reusing a key and IV in counter mode, on the other hand, is much worse. Because counter mode is a stream cipher, the plaintext is simply XORed with the key stream. If you happen to know the plaintext, you can recover the key stream:

$$K \oplus P \oplus P = K$$

"So what?" you might be thinking. "Who cares if they can get the key stream? If they already know the plaintext, why do we care?"

The problem is, under many circumstances an attacker might know some or all of the contents of one of your plaintext messages. If other messages are encrypted with the same key stream, the attacker can recover those messages too. Bad, bad, bad.

19.3 The credit card scenario

Let's explore this idea a little further. Suppose that you buy something for $100.00 with a credit card at a store. Let's assume a simplified version of the world where the card reader sends a message to your bank to authorise the purchase, protected by only AES-CTR encryption.

BOOK PAGE 93 – 94

Imagine that the message to the bank from the credit card reader is XML that looks like this:

the purchase message

```
<XML>
 <CreditCardPurchase>
  <Merchant>Acme Inc</Merchant>
  <Buyer>John Smith</Buyer>
  <Date>01/01/2001</Date>
  <Amount>$100.00</Amount>
  <CCNumber>555-555-555-555</CCNumber>
 </CreditCardPurchase>
</XML>
```

The store creates this message, encrypts it, and sends it to the bank. In order to communicate, the store and the bank must share a key. If the programmers who wrote the code were lazy and negligent, they may have created a system with a constant key and IV that are reused on every message.

19.4 Listing 3-10 AES-CTR for a Store

Listing 3-10. AES-CTR for a Store

```
1  # ACME generates a purchase message in their storefront.
2  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
3  from cryptography.hazmat.backends import default_backend
4
5  # WARNING! Never do this. Reusing a key/IV is irresponsible!
6  preshared_key = bytes.fromhex('00112233445566778899AABBCCDDEEFF')
7  preshared_iv = bytes.fromhex('00000000000000000000000000000000')
8
9  purchase_message = b"""
10 <XML>
11  <CreditCardPurchase>
12   <Merchant>Acme Inc</Merchant>
13   <Buyer>John Smith</Buyer>
14   <Date>01/01/2001</Date>
15   <Amount>$100.00</Amount>
16   <CCNumber>555-555-555-555</CCNumber>
17  </CreditCardPurchase>
18 </XML>
19 """
20
21 aesContext = Cipher(algorithms.AES(preshared_key),
22           modes.CTR(preshared_iv),
23           backend=default_backend())
24 encryptor = aesContext.encryptor()
25 encrypted_message = encryptor.update(purchase_message)
```

Line(s)	What that line actually does
5 – 7	The fatal lines. **preshared** means fixed at build time and never changed, and the IV is all zeros. Every message the store ever sends will use the identical key stream.

9 – 19	The message, hard-coded for simplicity. For simplicity the purchase message was included in the code; feel free to change it to accept a file or command-line flags that set the buyer's name, purchase price and so forth. You probably ought to also write the encrypted message to a file.
21 – 25	Ordinary, correct-looking CTR code. Nothing here is wrong the vulnerability is entirely in lines 6 and 7.

19.5 The attack

Back to our scenario. If you are trying to crack this system, you can spend $100.00 at this store, then tap the line and intercept the purchase message transmitted to the bank. If you do this, how much of the plaintext message do you know? You know all of it! You know who made the purchase, you know the amount of the purchase, you know the date, and you know your own credit card number.

That means that you can recreate the plaintext message, XOR it with the ciphertext, and recover key stream material. Because the merchant is reusing the same key and IV for the next customer, you can trivially decrypt the next customer's message and read the contents. Oops. We feel a news story about a data breach coming on.

EXERCISE 3.15 RIDING THE KEYSTREAM

Put into practice this keystream-stealing attack. That is, encrypt two different purchase messages using the same key and IV. "Intercept" one of the two messages and XOR the ciphertext contents with the known plaintext. This will give you a keystream. Next, XOR the keystream with the other message to recover that message's plaintext. The message sizes may be a little different, but if you're short some keystream bytes, recover what you can.

The whole attack is four lines

```
1 keystream = xor_bytes(known_plaintext, intercepted_ciphertext)
2 victim_plaintext = xor_bytes(keystream, victim_ciphertext)
3 print(victim_plaintext)
4 # the attacker never needed the key
```

BOOK PAGE 95 – 96

19.6 Even without knowing any plaintext

Even if the attacker does not know any of the plaintext and cannot recover a key stream, he or she can still take advantage of messages encrypted with the same key and IV pair. If you have two messages encrypted with the same key stream, you can do the following trick, where K is the key stream:

Step	Expression
given	$c_1 = m_1 \oplus K$
given	$c_2 = m_2 \oplus K$
XOR them	$c_1 \oplus c_2 = (m_1 \oplus K) \oplus (m_2 \oplus K)$
regroup	$c_1 \oplus c_2 = m_1 \oplus m_2 \oplus K \oplus K$
K cancels	$c_1 \oplus c_2 = m_1 \oplus m_2$

What do you get out of having the XOR of the two plaintext messages? Is that readable? It depends. Because plaintext messages often have structure, private data is often extractable or guessable.

19.7 Why the XOR of two plaintexts leaks so much

Take these made-up purchase messages from our example. If you XORed two such messages together, what could you learn? First of all, any parts that overlap exactly simply reduce to 0. Instantly, you know where the messages are the same and where they diverge. If the attacker were lucky enough that two messages had the same length of name for the buyer, the amount fields would line up as well. This field yields a lot of information when the two are XORed together because there are so few legal characters for this field the digits 0 to 9 and the decimal point. The XOR of the ASCII characters for the digits leaves open only a few possibilities.

For example, there are only two pairs of digits for which the XOR of their ASCII values is 15. These are “7” and “8” (ASCII 55 and 56) and “6” and “9” (ASCII 54 and 57). So, if we know that we have the XOR of two purchase amount field digits and the XOR value is 15, then the two messages each have one of these two pairs of numbers. That is only four possibilities, which will not be very difficult for an attacker to figure out under most circumstances.

verify it yourself

```
>>> ord('7') ^ ord('8')
15
>>> ord('6') ^ ord('9')
15
```

19.8 Full-duplex: the trap you will actually fall into

You might be surprised how often this vulnerability can show up if you are not careful. One simple example is full-duplex messages. If you have two parties that want to send encrypted messages to each other, they must

not use the same key and IV to encrypt each side of the connection. Each side's encryption must be independent of the other.

If you think about how CBC and CTR mode work, this will be pretty obvious. If you are going to write messages in both directions, each side needs a separate read key and write key. The read key of the first party will be the write key of the second and vice versa. This way, different messages will not be written under the same key and IV pair.

FOOTNOTE 13

Technically, the two parties could use the same key, provided the IVs were different. However, there are a number of ways in practice that IVs might intentionally or accidentally overlap, so it is typically recommended to use different keys and not rely on having different IVs.

EXERCISE 3.16 SIFTING THROUGH XOR

XOR together some plaintext messages and look around for patterns and readable data. There is no need to use any encryption for this; just take some regular, human-readable messages and XOR the bytes. Try human-readable strings, XML, JSON, and other formats. You may not find a lot that is instantly decipherable, but it's a fun exercise.

BOOK PAGE 96 – 98

20. Exploiting Malleability

Some aspects of cryptography are unintuitive at first. For example, an enemy can fail to read a confidential message while still being able to change it in meaningful, deceptive ways. In this section, we will experiment with altering encrypted messages while not being able to read them.

Counter mode is a really good encryption mode for all of the reasons described earlier. At the risk of being too repetitive, however, it only guarantees confidentiality. In fact, because it is a stream cipher, it is trivial to change a small part of the message without changing the rest of it. In counter mode, if an attacker modifies one byte of the ciphertext, it only affects the corresponding byte of plaintext. While that one byte of plaintext will not decrypt correctly, the remaining bytes will remain intact.

Cipher block chaining mode is different, because a change to a single byte of ciphertext will affect all subsequent blocks.

EXERCISE 3.17 VISUALIZING CIPHERTEXT CHANGES

To better understand the difference between counter mode and cipher block chaining mode, go back to the image encryption utility you wrote previously. Modify it to first encrypt and then decrypt the image, using either AES-CBC or AES-CTR as the mode. After decryption, the original image should be completely restored.

Now introduce an error into the ciphertext and decrypt the modified bytes. Try, for example, picking the byte right in the middle of the encrypted image data and setting it to 0. After corrupting the data, call the decryption function and view the restored image. How much of a difference did the edit make with CTR? How much of a difference did the edit make with CBC?

Hint: if you can't see anything, try an all-white image. If you still can't see it, change 50 bytes or so to figure out where the changes are happening. Once you find where the changes are happening, go back to changing a single byte to view the differences between CTR and CBC. Can you explain what's happening?

Mode	One flipped ciphertext byte damages…	What the image looks like
CTR	exactly one plaintext byte	a single wrong pixel you may need to hunt for it
CBC	that whole 16-byte block, plus one byte of the next block	one scrambled block, then the rest is fine

20.1 The attack that does not need reuse

To illustrate this concept of malleability, we are going to let our attacker know some of the plaintext of an encrypted message. This knowledge is going to allow them to change the message en route. What is different this time around is that this vulnerability is not dependent on a reused keystream.

If an attacker knows the plaintext behind a keystream-enciphered message, it is easy to extract the keystream from the ciphertext. If the keystream is reused, the attacker can decrypt all messages that used it. Even if it is not reused, the attacker can alter a message with known plaintext.

Let's revisit our encrypted purchase messages. Suppose that Acme's competitor, Evil LLC, wants to redirect this payment to themselves. They have a tap on the network connection coming out of Acme's store and can intercept and modify the message. When an encrypted form of this message comes along, even though they don't have the key and cannot decrypt it, they can strip out the original message parts that are known and replace them with their own chosen parts.

The part that Evil LLC wants to change is this part:

known and fixed in every payment message

```
<XML>
 <CreditCardPurchase>
  <Merchant>Acme Inc</Merchant>
```

That data is known and fixed in every payment message. To obtain the keystream, all Evil LLC has to do is XOR this data with the ciphertext. Once this part is XORed, they have the keystream for this many bytes. Then, they create their modified message:

the forgery

```
<XML>
 <CreditCardPurchase>
  <Merchant>Evil LLC</Merchant>
```

This message has the exact same size as the true message. Because AES-CTR is so malleable, it is easy to XOR this partial message with the extracted keystream and join it to the rest of the still-encrypted message.

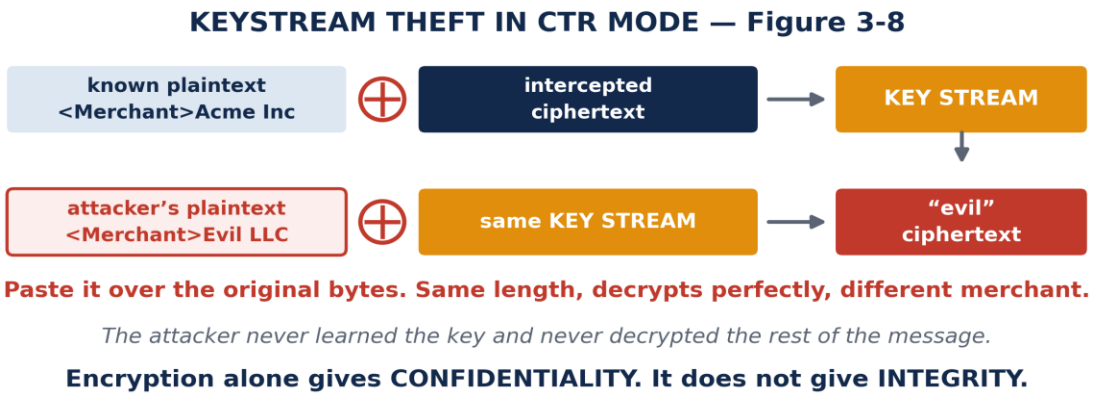


Figure 20.1 If an attacker knows the plaintext in CTR mode ciphertext, she can extract the keystream to encrypt her own evil message. (Book Figure 3-8.)

NOTICE WHY THE LENGTHS MATTER

"Acme Inc" and "Evil LLC" are both exactly eight characters. That is not a coincidence the authors got lucky with it is a requirement. In a stream cipher every byte position has its own key stream byte, so a substitution must be the same length or everything after it shifts out of alignment.

This is also why the attack is so easy to detect with a MAC and so impossible to detect without one.

BOOK PAGE 99

EXERCISE 3.18 EMBRACING EVIL

You work for (or own!) Evil LLC. Time to steal some payments from Acme. Start with one of the encrypted payment messages you created in the earlier exercises. Calculate the size of the header up through the identification of the merchant and extract that many bytes of the encrypted data. XOR the plaintext header with the ciphertext header to get the keystream. Once you have this, XOR the extracted keystream with a header identifying Evil LLC as the merchant. This is the "evil" ciphertext. Copy it over the bytes of the encrypted file to create a new payment message identifying your company as the recipient. Prove that it works by decrypting the modified file.

THE KEY LESSON OF THIS ENTIRE CHAPTER

Encryption is insufficient to protect data by itself.

In subsequent chapters, we will use message authentication codes, authenticated encryption, and digital signatures to ensure that data cannot be altered without disrupting communications.

21. Gaze into the Padding

While CBC mode is less susceptible to alteration than counter mode, it is by no means perfect in that regard. In fact, it is CBC's malleability that made one of the early versions of SSL vulnerable. Remember that CBC mode is a block-based mode and requires padding. An interesting error in the padding specification and the malleability of AES-CBC enabled attackers to execute a padding oracle attack and decrypt confidential data.

Let's create that attack right now. It is extremely interesting and educational.

For this little exercise, you will need to write your own padding functions; the ones in the cryptography module are too secure. Your functions will follow the very broken SSL 3.0 specification. Basically: N−1 bytes of anything, followed by a single byte that indicates the total length of the padding. Because padding was always required in that specification, it would be added even if the plaintext was a multiple of the block size. This will be important later.

21.1 Listing 3-11 SSLv3 Padding

Listing 3-11. SSLv3 Padding

```
1  def sslv3Pad(msg):
2    padNeeded = (16 - (len(msg) % 16)) - 1
3    padding = padNeeded.to_bytes(padNeeded+1, "big")
4    return msg+padding
5
6  def sslv3Unpad(padded_msg):
7    paddingLen = padded_msg[-1] + 1
8    return padded_msg[:-paddingLen]
```

Line(s)	What that line actually does

2	How many filler bytes are needed, *not counting* the length byte itself hence the `- 1`. If the message is already a multiple of 16, this evaluates to 15 and a whole extra block is added.
3	A neat trick: `padNeeded.to_bytes(padNeeded+1, "big")` produces a big-endian integer in `padNeeded+1` bytes, which means leading zero bytes followed by the value itself. The filler bytes are zeros and the final byte carries the length.
7	Reads the last byte and adds one, because the length byte is itself part of the padding.
8	Slices the padding off. **Note what this function never does: check the other 15 bytes.** That single omission is the entire vulnerability.

BOOK PAGE 100 – 101

21.2 Why the last block is the most malleable

The padding bytes in this scheme are completely ignored except for the last byte. It does not matter what the bytes are, so long as the last byte is correct. Padding goes at the end of a message, right? Guess which part of a CBC message is the most malleable.

The reason that the last part of the CBC message is more malleable is that it has no impact on any subsequent blocks. It can be changed without messing up anything else. Recall that CBC decryption starts out the same for every single block no matter where it is. The ciphertext block is decrypted by AES with the key. It is only after decryption that it is XORed with the ciphertext from the previous block.

This means that you could substitute any block from the CBC chain at the very end of the chain. It will get decrypted at the end just like it would in the middle or the beginning. After decryption, it is XORed with the ciphertext from the previous block.

21.3 The 1-in-256 opening

How is this helpful? Well, suppose that we are fortunate enough to have the original plaintext message be a multiple of 16 bytes long, the AES block length. Because we are using a padding scheme that always uses padding, there will be a full block of padding at the end. Since we do not care what bytes are in the padding except for the last one, we can correctly recover the entire message, even if we replace the last block, so long as the very last byte decodes to 15 the padding length when there is a full block of padding.

Explained another way: when there is a full block of padding at the end, 15 of the 16 bytes are completely ignored. It does not matter what they are. If we are going to try to fool the decryption, this is a great place to do it, because we only have to get one byte correct.

This small change only caring about the value of the last byte changes everything. It reduces brute-force guessing to something reasonable. Normally, if you wanted to guess a correct AES block, you would have to try all possible combinations of all 16 bytes. But now that we only care about the last byte, we only need to correctly guess one byte of data. One byte has 256 possible values, so if our last byte is randomly selected, then 1 out of 256 times it will correctly decrypt to 15.

What must be guessed	Possibilities	Feasible?
A whole 16-byte AES block	$2^{128} \approx 3.4 \times 10^{38}$	No not in the lifetime of the universe
Just the final padding byte	256	Yes a computer does this in milliseconds

You might protest that the data is not random. We are trying to decrypt a specific byte. Very true! But remember that in CBC we XOR the real plaintext with the ciphertext of the previous block. The ciphertext, at least for our purposes here, behaves like random data. For any given key/IV pair, the last byte of ciphertext that will be XORed with our plaintext byte has an equal chance of being any of the 256 possible one-byte values. If we are lucky, the “random” byte of ciphertext XORed with our plaintext byte will be 15.

21.4 Recovering the real plaintext byte

If the padding is accepted and decrypts to 15, we can use our knowledge of the previous ciphertext block to get the true plaintext byte.

Actually, recovering the plaintext byte is a little tricky and requires that we think through CBC decryption carefully. Remember that the last block of plaintext the true padding in the original message was XORed with the ciphertext from the second-to-last block. This intermediate data was encrypted by the AES algorithm. So, working backward, if we overwrite the final ciphertext block, the CBC operation will first run this block through the AES decryption operation to produce an intermediate value that is then XORed with the preceding ciphertext.

If the padding is accepted the last byte is 15 we know that the last byte of the intermediate value decrypted by AES is the XOR of 15 and the last byte of the previous ciphertext block. We, of course, have the ciphertext. Now, even without the AES key, we can simply compute the intermediate byte directly, by taking the XOR of 15 and the last byte of the second-to-last ciphertext block.

But the intermediate value is not the plaintext byte. Remember, we are decrypting an earlier ciphertext block. That ciphertext block is the AES encryption of the real plaintext XORed with the actual preceding ciphertext, or the IV if it is the first plaintext block. So when we recover the intermediate last byte, we still have to remove that mixed-in data with an appropriate XOR.

plaintext byte = 15 ⊕ last byte of penultimate ciphertext ⊕ last byte of the real previous block

BOOK PAGE 102

21.5 Listing 3-12 the oracle

Let's work on putting this into code. First, we need to define our oracle. In real life, the oracle was the SSLv3 server. If you sent it a message with bad padding, it would send you an error message that the padding was bad. That knowledge is all that is necessary to pull off this attack. For our code, we will just have an accept() method in an Oracle class that indicates whether the padding is valid, performing the same purpose as the server.

Listing 3-12. SSLv3 Padding Oracle

```
1  from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
2  from cryptography.hazmat.backends import default_backend
3
4  class Oracle:
5    def __init__(self, key, iv):
6      self.key = key
7      self.iv = iv
8
9    def accept(self, ciphertext):
10     aesCipher = Cipher(algorithms.AES(self.key),
11                        modes.CBC(self.iv),
12                        backend=default_backend())
13     decryptor = aesCipher.decryptor()
14     plaintext = decryptor.update(ciphertext)
15     plaintext += decryptor.finalize()
16     return plaintext[-1] == 15
```

Line(s)	What that line actually does
4 – 7	The oracle holds the key. This might seem a little weird we have the key and are using it to create the oracle. Just remember: we are simulating a vulnerable remote server, which would have its own key. The attack written below proceeds without any knowledge of the key used here.
9 – 15	Decrypts whatever it is handed. In the real world this is the SSLv3 server processing an incoming record.
16	THE ENTIRE VULNERABILITY, in one line. It returns a single boolean one bit of information about whether the padding was well formed. The server never intends to tell the attacker anything; it just wants to report an error. That one bit, repeated, hands over the whole message.

WHAT AN "ORACLE" MEANS IN CRYPTOGRAPHY

An oracle is anything that answers a question about a secret, however narrowly. It does not have to be a helpful error message a difference in timing, a different-length response, or even whether the connection is closed can all serve.

This is why modern protocols return one indistinguishable error for every kind of failure, and why cryptographic comparisons are written to take constant time. The lesson generalises far beyond SSLv3.

BOOK PAGE 103

21.6 Listing 3-13 getting one lucky byte

Once we have the oracle, it is a pretty easy function to see if we can get lucky and decode the last byte of an arbitrary block in the ciphertext.

Listing 3-13. Lucky SSLv3 Padding Byte

```
1  # Partial Listing: Some Assembly Required
2
3  # This function assumes that the last ciphertext block is a full
4  # block of SSLV3 padding
5  def lucky_get_one_byte(iv, ciphertext, block_number, oracle):
6    block_start = block_number * 16
7    block_end = block_start + 16
8    block = ciphertext[block_start: block_end]
9
10   # Copy the block over the last block.
11   mod_ciphertext = ciphertext[:-16] + block
12   if not oracle.accept(mod_ciphertext):
13     return False, None
14
15   # This is valid! Let's get the byte!
16   # We first need the byte decrypted from the block.
17   # It was XORed with second to last block, so
18   # byte = 15 XOR (last byte of second-to-last block).
19   second_to_last = ciphertext[-32:-16]
20   intermediate = second_to_last[-1]^15
21
22   # We still have to XOR it with its *real*
23   # preceding block in order to get the true value.
24   if block_number == 0:
25     prev_block = iv
26   else:
27     prev_block = ciphertext[block_start-16: block_start]
28
29   return True, intermediate ^ prev_block[-1]
```

21.7 Line by line

Line(s)	What that line actually does
5	Four parameters: the IV, the ciphertext, which block we are attacking, and the oracle to ask.
6 – 8	Slices out the target block the one whose last byte we want to learn.
11	The heart of the attack. `ciphertext[:-16]` drops the real final block and `+ block` pastes our target block in its place. The message is still exactly the same length, so the server will process it normally.
12 – 13	Ask the oracle. If the padding is rejected we were unlucky; return failure and try again with a fresh key/IV.

Line(s)	What that line actually does
19	`ciphertext[-32:-16]` is the penultimate block the one that will be XORed against our substituted block during decryption.
20	Because the oracle said yes, we know the final decrypted byte is 15. Undo that XOR to recover the byte AES produced internally. **This is the step that needs no key.**
24 – 27	The intermediate value still has the *real* previous block mixed into it. For block 0 that is the IV; otherwise it is the block immediately before the target.
29	One more XOR removes it, and out falls a genuine byte of plaintext.

To repeat: we are counting on the penultimate block being lucky. We have to be lucky enough that the last byte of the penultimate block will just happen to XOR with our intermediate byte to be 15. This luck that we are counting on is dependent on the key and IV chosen. For any given key/IV pair, there is a 1-in-256 chance that the penultimate block will accidentally XOR with our intermediate plaintext block to give us 15.

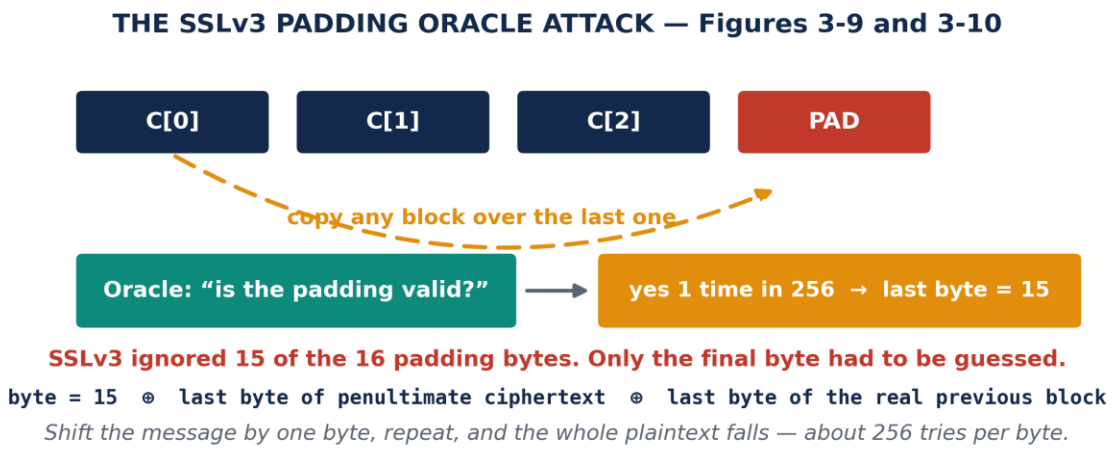


Figure 21.1 If the first 15 bytes of the padding block are ignored, we can substitute in the second-to-last block and see if the oracle tells us the padding is correct. If so, we can figure out the last byte in the previous block. (Book Figures 3-9 and 3-10.)

BOOK PAGE 104 – 106

21.8 One byte in 256 tries is that really useful?

Is that really all that useful? In the first place, we have to be lucky enough to have a full block of padding. In the second place, we only have a 1-in-256 chance of decoding a single byte. That does not seem terribly helpful.

Or does it?

Again, cryptography can be very counter-intuitive. Computers do not behave like we would expect them to, and that is where we get into trouble.

21.9 Why it was devastating in practice

While SSLv3 was busy protecting web traffic, it turned out there were a number of ways that a malicious advertisement could generate traffic to an SSL-encrypted web site. But because that advertisement was generating the traffic, its authors could control how long the encrypted message was. Thus, if the attacker was trying to decrypt an encrypted cookie, triggering a GET request of various lengths could control how long the overall message was.

It is still just one byte, right? The attacker can control the length of the message. Once one byte is decoded, it is pretty straightforward to increase the message length by 1 by inserting a byte earlier in the message, pushing the new byte into the last slot of the arbitrary block. Another 256 tries and that second byte will be decoded too. Wash, rinse and repeat.

Getting the full block of padding in this case really is not very difficult, as the malicious requester could put arbitrary data in the GET request.

And it is nothing for a computer to make 256 requests over a network. Note that, in the SSLv3 context, the client and server are going to use different keys with every connection for good reason, as we have seen. This means that on each connection the ciphertext will be different. So if the attacker sends 256 requests, the penultimate block will be different each time, providing a new opportunity to be lucky and have the right "random" number that will provide the needed 15.

> **THE BITTER IRONY**
>
> Re-keying every connection is excellent security practice it is exactly what Section 19 told you to do. Here that same good practice is what makes the attack work, because every fresh connection is a fresh roll of the 256-sided die.
>
> This is the real-world POODLE attack, disclosed in 2014. It is why SSLv3 is switched off everywhere today.

EXERCISE 3.19 RESISTANCE IS FUTILE

Finish the code for the padding oracle attack. We've given you the major pieces, but it will still take some work to put everything together. We will do a few things to try and simplify as much as possible. First, pick a message that is exactly a multiple of 16 bytes in length (the AES block size) and create a fixed padding to append. The fixed padding can be any 16 bytes as long as the last byte is 15 (that's the whole point of the exercise, right?). Encrypt this message and pass it to the oracle to make sure that code is working.

Next, test recovering the last byte of the first block of the message. In a loop, create a new key and IV pair (and a new oracle with these values), encrypt the message, and call the lucky_get_one_byte() function, setting block number to 0. Repeat the loop until this function succeeds and verify that the recovered byte is correct. Note that, in Python, an individual byte isn't treated as a byte type but is converted to an integer.

The last step in order to decode the entire message is to be able to make any byte the last byte of a block. Again, for simplicity, keep the message being encrypted a perfect multiple of 16. To push any byte to the end of a block, add some extra bytes at the beginning and cut off an equal number at the end. You can now recover the entire message one byte at a time!

EXERCISE 3.20 STATISTICS ARE ALSO FUTILE

Instrument your padding oracle attack in the previous exercise to calculate how many guesses it took to fully decrypt the entire message and calculate an average number of tries per byte. In theory, it should work out to about 256 tries per byte. But you're probably working with such small numbers that it will vary widely. In the authors' tests on a 96-byte message, their averages varied between about 220 guesses per byte and 290 guesses per byte.

Once again, encryption is about confidentiality, and confidentiality is simply not enough to solve all security problems. In subsequent chapters we will learn how to combine confidentiality and integrity to solve a larger class of problems.

BOOK PAGE 107

22. Weak Keys, Bad Management

To conclude this chapter, let's briefly discuss keys. Hopefully, it has already become very clear to you how important keys are.

In almost all cryptographic systems, key management is the hardest part. It can be difficult to generate good keys, to share keys, and to manage keys afterward keeping them secret, updating them, or revoking them. For now, we will focus on key generation.

22.1 Keys must come from good randomness

Keys must be drawn from good sources of randomness. We mentioned randomness once already in this chapter in a brief aside, but let's take a second look. For example, the following code is really wrong.

WRONG do not do this

```
1 import random
2 key = random.getrandbits(16, "big")
```

The random package is a pseudo-random number generator, and not even a good one at that. Pseudo-random generators are deterministic, generating numbers that appear random to humans, but are always the same given a known seed value. Default seeds used to be based on the system time. This may seem reasonable, but it means that if the attacker can guess when the random number generator was seeded, they can completely predict all the random numbers produced.

About the only way to make this worse is to hard-code the key or the seed, which is effectively the same thing.

EVEN WORSE

```
1 import random
2 # Set the random number generator seed to 0.
3 r = random.Random(0)
4 key = r.getrandbits(16, "big")
```

This code will produce the same "random" numbers on every run of the program. This can sometimes be useful for testing, but you must not leave it in production code.

22.2 What to use instead

Although Python's default seeding is no longer quite so predictable, it is not suitable for generating secrets like passwords. Instead, always pull from os.urandom() or, if using Python 3.6 or later, secrets.SystemRandom(). Under most circumstances, this is enough randomness. If you need something stronger, you might need to use different hardware and should consult an expert cryptographer.

Source	Suitable for keys?	Why
random.random(), random.getrandbits()	NEVER	Deterministic; fully predictable from the seed

Source	Suitable for keys?	Why
random.Random(0)	NEVER	Fixed seed identical output on every run
os.urandom(n)	Yes	Draws from the OS cryptographic entropy pool
secrets module (Python 3.6+)	Yes	Purpose-built wrapper over the same source

BOOK PAGE 108

22.3 Keys derived from passwords

In some deployments, a key is not pulled from random numbers. Instead, it is derived from a password. If you are going to derive a key from a password, the password needs to be very secure. In the previous chapter you learned about brute-force attacks, and all of those lessons apply here.

Let's get a feel for the difference in difficulty of guessing a key in these scenarios. How long would it take to try every possible 128-bit random key? How many tries is that? There are 2^{128} different 128-bit keys. That is this many:

340,282,366,920,938,463,463,374,607,431,768,211,456

If your key is derived from a five-digit PIN number, though, you have reduced it to 99,999. It is true that very few passwords will be as hard to brute force as a truly random 128-bit key. After all, you would need to have a password composed of about 20 random characters to require the same kind of brute-force effort as a 128-bit key. But still, 99,999 is just begging a computer to accept your challenge. You can do better than that.

Key source	Number of possibilities	Verdict
Random 128-bit key	$2^{128} \approx 3.4 \times 10^{38}$	Out of reach
~20 random characters	comparable to 2^{128}	Acceptable, if truly random
Five-digit PIN	99,999	Cracked instantly

As a reminder, there are proven algorithms for deriving a key from a password. Make sure you use a good one. In the previous chapter, we used scrypt. There are others that some people feel are even better, such as bcrypt or Argon2. What makes a good derivation function? One characteristic is how long it takes. If someone picks a weak password such as "puppy1", it will not take the attacker long to figure it out. It might be possible, however, to make it take too long if the derivation function is slow.

In short, do not bother using a good cipher with a bad key. Make sure that your keys are securely generated and adequately resistant to abuse by a determined adversary.

EXERCISE 3.21 PREDICTING TIME-BASED RANDOMNESS

Write an AES encryption program (or modify one of the others you've written for this chapter) that uses the Python random number generator to generate keys. Use the seed method to explicitly configure the generator based on the current time using time.time() rounded to the nearest second. Then use this generator to create a key and encrypt some data. Write a separate program that takes the encrypted data as input and tries to guess the key. It should take a minimum time and a maximum time as a range and try iterating between these two points as seed values for random.

DO THIS EXERCISE IT IS THE MOST SOBERING ONE IN THE CHAPTER

AES-256 has 2^{256} keys. Seed the generator with the current second and the attacker only has to search the number of seconds in the window they think you ran the program perhaps a few thousand values. A whole day is only 86,400.

The cipher was never broken. The key space was. This is the single most common way that real deployed cryptography fails.

BOOK PAGE 109

23. Other Encryption Algorithms

In this chapter, we have focused exclusively on AES encryption. There are good reasons for this. AES is the most popular symmetric cipher currently in use. It is used in network communications as well as storing data on disk. And as we will see in Chapter 7, it is the basis for several advanced AEAD (authenticated encryption with associated data) constructions.

However, there are other symmetric key ciphers that can be used. Here are a few that are supported by the cryptography library:

- Camellia
- ChaCha20
- TripleDES
- CAST5
- SEED

23.1 A warning about legacy support

Even though we are always encouraging you to use a well-tested, well-respected third-party library, be aware that libraries often include support for less desirable algorithms for legacy support. In this list of algorithms supported by cryptography, a few ciphers are already known to be insecure and are being phased out.

For example, while DES is not included in the cryptography library's ciphers GOOD! DES is VERY BAD! the module does include 3DES (TripleDES). While 3DES is not as broken as DES, it should be retired as soon as possible. CAST5 fits in this same category.

Another cipher supported by cryptography is Blowfish. This algorithm is also not recommended for use, and its stronger successor, Twofish, is not available in the current cryptography implementation.

Cipher	Status	Note
AES	Use this	The focus of this chapter and the basis of AEAD in Chapter 7
ChaCha20	Good modern alternative	A stream cipher; widely used in TLS alongside AES
Camellia	Acceptable	Comparable to AES; common in Japanese standards
TripleDES (3DES)	Retire ASAP	Not as broken as DES, but obsolete
CAST5	Retire ASAP	Same category as 3DES
Blowfish	Not recommended	Successor Twofish is not in the library
DES	VERY BAD	Not supported by cryptography at all and rightly so

THE LESSON HIDDEN IN THIS LIST

Cipher	Status	Note

A library exposing an algorithm is not an endorsement of it. Libraries carry legacy ciphers so that old systems can still be read, and the API gives you no warning at all when you pick a bad one.

"It was in the library so it must be fine" is exactly the reasoning that led Bob to ECB mode on page 54. The chapter opens and closes on the same mistake.

24. finalize()

This chapter covered a lot of material, and we barely scratched the surface. Perhaps the most important principle that you can take away from this chapter is that cryptography is usually far more complicated than perhaps it first seems. The different modes of operation we reviewed have different strengths and weaknesses, some of which we explored by example. We found that even how we approach the APIs to cryptographic operations can have a significant impact on security.

Hopefully, this lesson reinforced the YANAC principle You Are Not A Cryptographer… yet! Please remember that these exercises are introductory and educational. Please do not go copying this code into production, and do not use the introductory knowledge you have gained to write security-critical operations. Do you really want to risk people's personal information, financial information, or other sensitive data on your newly developed skills?

At the same time, after just one chapter on encryption, you have a broader view of what that word even means. The next time you hear "protected by AES 128-bit encryption", you might wonder whether they are using CTR, CBC, or (heaven forbid!) ECB mode. You might also wonder if they are using their encryption correctly, because you have already experienced some of the ways often unexpected that symmetric encryption can be broken.

Yes, you have taken your first steps into a cryptography world. Are you ready to take a few more? Then let's talk about asymmetric encryption!

Appendix A Chapter Summary at a Glance

Section (book pages)	The one thing to remember
Let's Scramble! (53)	Symmetric = one key for both directions. AES is the standard.
Listing 3-1 (54–55)	Cipher(algorithm, mode, backend) → .encryptor() / .decryptor().
The b'' mystery (55–56)	update() buffers until a full 16-byte block exists.
What Is Encryption, Really? (57)	Confidentiality, integrity, authentication. This chapter buys only the first.
AES: A Symmetric Block Cipher (58–59)	Block size is always 128 bits; key may be 128, 192 or 256.
The three modes (60)	ECB (never), CBC (legacy), CTR (recommended). GCM comes later.
ECB Is Not for Me (60–63)	Raw AES. Its only good use is the NIST known-answer tests.
Identical blocks (64–67)	Same plaintext block → same ciphertext block. Kerckhoff's principle.
The image experiment (68–70)	The TOP SECRET bitmap is still readable after real AES-ECB.
Spontaneous Independence (70–71)	IVs make output unrepeatable. The IV is public; the key is not.
XOR (71–72)	(A ⊕ B) ⊕ B = A. Python's operator is ^.
CBC (73–75)	Encrypt: XOR then AES. Decrypt: AES then XOR. IV is always 16 bytes.
Proper Padding (75–79)	PKCS7 appends n bytes of value n. Nothing is padded until finalize().
Listing 3-7 vs 3-8 (80–85)	Never hand the same key/IV pair to an encryptor twice.
Cross the Streams (86–91)	CTR turns AES into a stream cipher. Encryption = decryption.
Key and IV Management (91–96)	Reuse in CBC leaks patterns; in CTR it hands over the key stream.
Exploiting Malleability (96–99)	Known plaintext lets an attacker rewrite a message she cannot read.
Gaze into the Padding (99–106)	One bit of padding feedback decrypts everything, 256 tries per byte.
Weak Keys (107–108)	random.seed(time) destroys a 2^{128} key space. Use os.urandom.
Other Algorithms (109)	A library shipping a cipher is not an endorsement of it.
finalize() (109–110)	Encryption alone is not security. Chapters 4 to 8 supply the rest.

Appendix B Glossary of Chapter 3 Terms

Term	Meaning
Symmetric encryption	The same key encrypts and decrypts.
AES / Rijndael	The Advanced Encryption Standard; a symmetric block cipher.
Block cipher	Operates on fixed-size blocks. AES blocks are 16 bytes.
Stream cipher	Operates a byte at a time; needs no padding.
Mode of operation	How a block cipher is applied to a message longer than one block.
ECB	Electronic code book. Each block encrypted independently. Never use it.
CBC	Cipher block chaining. Each block XORed with the previous ciphertext.
CTR	Counter mode. AES encrypts a counter to make a key stream.
GCM	A modern authenticated mode, covered later in the book.
IV	Initialisation vector. Public, 16 bytes, never reused with one key.
Nonce	The same idea in CTR mode: the starting counter value.
XOR (⊕)	Bitwise exclusive-or. Its own inverse.
Key stream	The pseudo-random bytes CTR mode XORs with the plaintext.
One-time pad	XOR with a truly random key as long as the message. Unbreakable, impractical.
Padding	Filler bytes added so the message is a whole number of blocks.
PKCS7	A padding scheme: n bytes, each holding the value n.
ANSI X.923	A padding scheme: zeros, with the length in the final byte.
KAT	Known answer test. Published input/output pairs that validate an implementation.
Kerckhoff's principle	The system must be secure even if everything but the key is public.
Chosen-plaintext attack	The attacker gets you to encrypt data of their choosing.
Malleability	The ability to change a ciphertext meaningfully without reading it.
Padding oracle	Anything that reveals whether decrypted padding was valid.
POODLE	The real-world 2014 attack this chapter reconstructs.
hazmat	The cryptography module's "hazardous materials" layer of low-level primitives.
YANAC	You Are Not A Cryptographer… yet.

Appendix C Self-Assessment and Viva Questions

C.1 Short-answer questions

1. Why did aesEncryptor.update(b'alice') return b''? What exactly was in the buffer at that moment?
2. State the three security properties and say which one AES alone provides.
3. The AES key can be 16, 24 or 32 bytes. Why is the CBC IV always exactly 16 bytes?
4. Why is the TOP SECRET image still readable after ECB encryption, even though AES is unbroken?
5. Write out the CBC encryption and decryption formulas and state which order the XOR and AES operations occur in each.
6. What is the only legitimate use for ECB mode mentioned in this chapter?
7. Explain why Listing 3-6 produced five outputs from three inputs.
8. Listing 3-7 looks cleaner than Listing 3-8. What exactly is wrong with it?
9. Why is reusing a key/IV pair worse in CTR mode than in CBC mode? Give the one-line algebra.
10. Why must “Evil LLC” be exactly the same length as “Acme Inc” in the malleability attack?
11. In the padding oracle attack, how much information does the oracle return per query, and why is that enough?
12. Why does re-keying every connection make the POODLE attack easier rather than harder?
13. An AES-256 key has 2^{256} possibilities. Explain how seeding random with time.time() reduces that to a few thousand.

C.2 Practical (laboratory) tasks

1. Exercise 3.1 explore update() until you can predict exactly when output appears.
2. Exercise 3.2 rewrite the Chapter 1 cipher application to use AES.
3. Exercises 3.3 and 3.4 research DES, 3DES, RC4 and Twofish.
4. Exercise 3.5 parse a NIST .rsp file and validate your library against every vector.
5. Exercises 3.6 and 3.7 build the EATSA messages, then splice two ciphertexts together.
6. Encrypt a bitmap in ECB mode and paste both images into your lab record.
7. Exercises 3.8 and 3.9 XOR practice and the masking exercise.
8. Exercises 3.10, 3.11 and 3.12 deterministic output, the CBC image, and hand-built CBC.
9. Exercises 3.13 and 3.14 build counter mode from ECB, then parallelise the key stream.
10. Exercises 3.15 and 3.16 steal a key stream, and XOR two plaintexts to see what leaks.
11. Exercises 3.17 and 3.18 corrupt one byte in CBC and CTR, then forge the Evil LLC payment.
12. Exercises 3.19 and 3.20 complete the padding oracle attack and measure the guesses per byte.
13. Exercise 3.21 break a time-seeded key and record how long the search took.

C.3 Discussion questions

- A vendor advertises “bank-grade 256-bit AES encryption”. List five questions you would need answered before believing your data is safe.
- Your project team wants one key hard-coded in the app so the client and server can talk. Explain what an attacker with a decompiler does next.
- Explain to a non-technical manager why the TOP SECRET image is still readable, without using the words “block” or “mode”.

Appendix D Common Errors and How to Fix Them

Symptom	Likely cause	Fix
update() keeps returning b''	Fewer than 16 bytes have been supplied so far.	Keep feeding data, or call finalize().
ValueError on finalize()	An incomplete block is still buffered in a block mode.	Use a padder, or supply whole blocks.
Decryption returns garbage in hand-built CBC	XOR applied before AES during decryption.	Decrypt with AES first, THEN XOR with the previous ciphertext.
Same plaintext gives the same ciphertext	The key/IV pair is being reused.	Generate a fresh IV for every message.
ValueError: Invalid IV size	IV length does not match the block size.	AES IVs are always 16 bytes, whatever the key size.
Messages come back concatenated, not separate	One context was used for all messages.	Expected behaviour use one context per message, or frame the output yourself.
Unpadder returns b'' for a full block	The unpadder reserves the last block.	Expected behaviour call finalize() to flush it.
Encrypted image still shows the picture	ECB mode.	Use CBC or CTR. This is the whole point of Section 10.
The decrypted file is corrupt from one point onward	A byte was altered in a CBC ciphertext.	Expected CBC propagates damage forward. Add a MAC to detect it.
Keys are identical on every program run	random was seeded with a fixed or time-based value.	Use os.urandom() or the secrets module.
TypeError: Unicode-objects…	A str was passed where bytes were required.	Use b'' literals or .encode('utf-8').

Reference

S. J. Nielson and C. K. Monson, Practical Cryptography in Python: Learning Correct Cryptography by Example. Berkeley, CA: Apress, 2019. Chapter 3, “Symmetric Encryption: Two Sides, One Key”, pp. 53–110. https://doi.org/10.1007/978-1-4842-4900-0_3